

Degree Project in Computer Engineering

First cycle, 15 credits

Analysis of Latency Contributions in an XR Sensor Node–Server System

A Quantitative Breakdown of Latency Contributions Across Modules in an XR Sensor Node–Server System

ANDRÉ TISLJAREC
JACOB EMANUELSSON

Analysis of Latency Contributions in an XR Sensor Node–Server System

A Quantitative Breakdown of Latency Contributions Across Modules in an XR Sensor Node–Server System

ANDRÉ TISLJAREC

JACOB EMANUELSSON

Degree Programme in Computer Engineering
Date: June 15, 2026

Supervisor: Ludvig Lindström, Mattias Wildeman
Examiner: Isaac Skog

Host company: Ericsson
Swedish title: Analys av Latenssbidrag i ett XR-Sensornod-Serversystem
Swedish subtitle: En Kvantitativ Analys av Latensbidrag Från Olika Moduler i ett XR-Sensornod-Serversystem

© 2026 André Tisljarec and Jacob Emanuelsson

Abstract

Real-time situational awareness systems for firefighters require sensor data to be collected, processed, and transmitted with low latency. This is relevant for future systems using Extended Reality (XR) user equipment and cellular networks, where data from multiple sensors may be shared with remote systems to support decision-making. However, the end-to-end latency of a multi-sensor edge system is difficult to evaluate because delay can be introduced at several stages, including sensor data acquisition, local data processing, communication interfaces, encoding, and network transmission.

This thesis is conducted in collaboration with Ericsson, within the Network-Enabled eXchange of critical information in Systems-of-Systems (NEXSoS) project. It investigates the latency contribution from different modules in a system consisting of a Raspberry Pi-based multi-sensor node with 5G transmission capabilities and a cloud server. The sensor node includes a Red Green Blue (RGB) camera, an Infra Red (IR) camera, a Light Detection and Ranging (LiDAR) sensor, and an Inertial Measurement Unit (IMU). The work combines a design science approach with experimental evaluation. A sensor node prototype was implemented, the total end-to-end latency was decomposed into feasible and non-feasible measuring points, where feasible points were measured. These points included sensor acquisition, processing, transmission latency, and network transmission latency over both Ethernet and 5G. The network transmission was implemented using User Datagram Protocol (UDP).

The results of the experiments show that the latency contribution from the network module and the sensing modules varied significantly. The dominant latency contributor being the infrared camera pipeline, where the average total latency was approximately 75 ms, overhead from onboard IR processing remained relatively low at 0.26 ms. The other sensor modules, IMU, LiDAR, and the RGB camera demonstrated a lower latency contribution. IMU and LiDAR averaged 11.1 ms and 1.84 ms respectively. The RGB camera had a latency of approximately 10.7 ms.

The network's latency contribution varied when experiments were conducted on wired versus wireless transmission. Ethernet communication achieved a median Round-Trip Time (RTT) of 3.9 ms, whereas 5G transmission exhibited a higher median RTT of 28.5 ms with occasional outliers.

The thesis demonstrates that the developed sensor node prototype can be used to measure latency contribution from different stages in a real-time sensor

pipeline.

Keywords

IoT, Sensor node, 5G, Latency, XR, Packet Loss

Sammanfattning

Realtidsbaserade system för situationsmedvetenhet för brandmän kräver att sensordata samlas in, bearbetas och överförs med låg latens. Detta är relevant för framtida system som använder XR-användarenheter och mobilnät, där data från flera sensorer kan delas med fjärrsystem för att stödja beslutsfattande. Den end-to-end-latens som uppstår i ett edge-system med flera sensorer är dock svår att utvärdera eftersom fördröjning kan introduceras i flera steg, inklusive sensorinsamling, lokal bearbetning, kommunikationsgränssnitt, kodning och nätverksöverföring.

Denna avhandling som genomförts i samarbete med Ericsson, inom NEXSoS-projektet avser att kvantifiera latensbidraget från olika modular i ett system bestående av en Raspberry Pi-baserad multisensor-nod med 5G-överföringskapacitet och en molnserver. Sensornoden inkluderar en RGB-kamera, en IR-kamera, en LiDAR-sensor och en IMU. Arbetet kombinerar en design naturvetenskaplig-metod med experimentell utvärdering. En prototyp av sensornoden implementerades, och den totala latensen bröts ner i mätbara och icke-mätbara delsteg, där de mätbara delstegen kunde utvärderas experimentellt. Dessa inkluderade sensorinsamling, överföringslatens samt nätverkslatens över både Ethernet och 5G. Nätverksöverföringen implementerades med UDP, där kameraramar delades upp i paket och bekräftades av en mottagande fjärrnod.

Resultaten från experimenten visar att latensbidraget från nätverksmodulen och sensormodulerna varierar betydligt. Den dominerande latenskällan är den infraröda kamerans pipeline, där den totala genomsnittliga latensen var cirka 75 ms, medan overhead från lokal IR-bearbetning var relativt låg (0,26 ms). De övriga sensormodulerna, IMU, LiDAR och RGB-kameran uppvisade lägre latensbidrag. IMU och LiDAR hade genomsnittliga latenser på 11,1 ms respektive 1,84 ms, medan RGB-kameran låg på cirka 10,7 ms.

Nätverksbidragande latens varierade mellan trådbunden och trådlös överföring. Ethernetkommunikation uppvisade en median-RTT på 3,9 ms, medan 5G-överföring hade en högre median-RTT på 28,5 ms med vissa utstickare. Paketförlusten var försumbar i både trådbundna och trådlösa tester.

Avhandlingen visar att den utvecklade sensornodens prototyp kan användas för att mäta latensbidrag från olika steg i en realtidsbaserad sensorkedja. Sammantaget domineras latensen av sensorns insamlings- och bildbehandlingspipeline samt varierande trådlös kommunikation, medan beräkning och trådbunden överföring bidrar med relativt liten och stabil overhead.

Nyckelord

IoT, sensornod, 5G, latens, paketförlust

Acknowledgments

We would like to thank Mattias Wildeman and Björn Skubic for giving us the opportunity be a small part of the NEXSoS project. And also for all of their help and valuable input which helped us throughout the project. We would also like to thank our supervisor Ludvig Lindström for providing us continuous feedback throughout the project.

AI has been used for generating the figures in the report, for grammar correction, developing and debugging software.

Stockholm, June 2026
André Tisljarec Jacob Emanuelsson

Contents

1 Introduction **1**
1.1 Background . . . 2
1.2 Purpose . . . 3
1.3 Problem and Definition . . . 3
1.4 Goals . . . 4
1.5 Research Methodology . . . 4
1.6 Delimitations . . . 4
1.7 Structure of the Thesis . . . 5

2 Background **6**
2.1 Internet of Things . . . 6
2.1.1 Sensor Node . . . 7
2.1.2 Edge and Cloud Processing . . . 7
2.2 Light Detection and Ranging . . . 7
2.3 Inertial Measurement Unit . . . 8
2.4 Overview of Latency . . . 8
2.5 Device Level Communication Protocols . . . 9
2.5.1 USB Video Class . . . 10
2.6 Network Transmission Protocols . . . 10
2.6.1 Transmission Control Protocol . . . 11
2.6.2 User Datagram Protocol . . . 11
2.6.3 Constrained Application Protocol . . . 12
2.7 Fifth Generation of Cellular Network . . . 12
2.8 Packet Loss . . . 12
2.9 Bandwidth . . . 13
2.10 Kubernetes . . . 13
2.11 Related Work . . . 14
2.11.1 Latency in Video Transmission . . . 14
2.11.2 Packet Loss in Video Transmission . . . 14

2.11.3 5G Networks for XR Use Cases . . . 15
2.11.4 Motion-To-Photon Tolerance Latency . . . 16

3 Methods 18
3.1 Research Process . . . 19
3.2 Data Collection . . . 19
3.3 Experimental design and Planned Measurements . . . 20
3.3.1 Experimental Setup . . . 21
3.3.2 Hardware and Software to be Used . . . 21
3.3.3 Expected Phase Delay under Asynchronous Sampling 21
3.3.4 Sample Size . . . 22
3.4 Assessing Reliability and Validity of the Data Collected . . . 22
3.4.1 Validity and Reliability of Method . . . 22
3.4.2 Validity and Reliability of Data . . . 23
3.5 Planned Data Analysis . . . 24
3.5.1 Data Analysis Technique . . . 24
3.6 Evaluation Framework . . . 24

4 Implementation 25
4.1 Construction . . . 25
4.2 Experiment Setup . . . 26
4.2.1 RGB Camera . . . 26
4.2.2 LiDAR . . . 29
4.2.3 IMU . . . 29
4.2.4 IR Camera . . . 31
4.2.5 Cloud server . . . 33
4.2.6 Network . . . 33

5 Results and Analysis 35
5.1 Sensor and Transmission Results . . . 35
5.1.1 LiDAR . . . 35
5.1.2 IMU . . . 36
5.1.3 IR Camera . . . 37
5.1.4 Photon-to-frame-in-RAM pre ISP . . . 38
5.1.5 Network Transmission via 5G . . . 38
5.1.6 Network Transmission via Ethernet . . . 41
5.2 Reliability Analysis . . . 42
5.2.1 LiDAR . . . 44
5.2.2 IMU . . . 45
5.2.3 IR Camera . . . 46

5.2.4 Photon-to-frame-in-RAM pre ISP Latency 47
5.2.5 Network Transmission 48
5.3 Validity Analysis 48
5.3.1 LiDAR 49
5.3.2 IMU 49
5.3.3 IR Camera 50
5.3.4 Photon-to-frame-in-RAM pre Image Signaling Processing (ISP) Latency 50
5.4 Network Transmission 51

6 Discussion **52**
6.1 Sample Size 52
6.2 LiDAR 53
6.3 IMU 54
6.4 IR Camera 55
6.5 Photon-to-frame-in-RAM pre ISP Latency 57
6.6 Network Transmission 58

7 Conclusions and Future Work **60**
7.1 Conclusion 60
7.2 Limitations 62
7.3 Future Work 62
7.4 Reflections 63

List of Figures

4.1 The picture shows the high level architecture design of the system. The four sensors are connected to the Raspberry Pi and handled differently depending on type. The 5G hat acts as the bridge between the device and the cloud. 26
4.2 This picture represented the testbed implemented to test a real-time event until it reaches memory in the device. The Raspberry Pi turns on a LED and times the time it takes after the LED is turned on until the event has arrived in device memory. 27
4.3 Figure shows the different stages that make up the total latency pipeline of photon-to-frame-in-RAM latency, for a camera sensor using rolling shutter. Latency contribution from stages such as t_{csi}, t_{dma} and $t_{readout}$ are unknown. While contribution from t_{exp} is known, t_{phase} has a known interval related to the frame period. t_{led} is the reference event used to timestamp the start of the measurement 28
4.4 The practical set up for the LiDAR is shown here. This setup is using a trigger signal from the Raspberry Pi and measure time between signal sent until data comes back. 30
4.5 This figure shows the sequence of events that happens when the test runs. The trigger signal is sent and when the LiDAR receives the signal it starts the measurement, then it sends data back. 30
4.6 The figure shows the sequence of events that happens in the test. This figure is a high level overview not an exact representation. t_0 represents when the Interrupt (INT) goes up, t_1 shows approximated when Raspberry Pi register the event. t_2 is when the Raspberry Pi start to read the data and t_3 is when it is finished reading. 31

4.7 This figure provides an overview of the measurement pipeline used in the test. The measured sections are the internal processing delay on the camera board and the delivery time from the processed frame leaving the board until it reaches the Raspberry Pi. 32
4.8 The figure shows the timeline of events that occur during the test. The frame is captured by the camera and right before the frame is being sent to the modem we start the clock. Sending it to the cloud service where it is then being processed and finally a Acknowledgment (ACK) is sent back and that is were the clock stops. The RTT is the total time from that a frame is being sent until an ACK is sent back. 34

5.1 Estimated USB Video Class (UVC) frame delivery delay from camera board Universal Serial Bus (USB) packetization to Raspberry Pi's memory reception over 200 frames. 38
5.2 Latency distribution Photon-to-frame-in-RAM pre ISP latency from 50 trails. 39
5.3 Mean latency decomposition Photon-to-frame-in-RAM pre ISP latency. Figure shows an estimation of latency contribution from phase offset and camera pipeline. Section 3.3.3 explains how the average phase offset is determined and how it impacts the overall latency. 40
5.4 Histogram of RTT for 199 frame pairs transmitted over 5G, excluding one frame pair affected by packet loss. The data is grouped into 5 ms intervals, with the majority of samples falling between 25 ms and 30 ms. 42
5.5 Packetloss for each frame on each camera, total loss of one frame. This amounts to a package loss of 0.025% over both cameras. Packet loss occurred at frame 11, transmitted over 5G. 43
5.6 This figure shows the RTT for each frame pair, transmitted over 5G. It shows the occurrences of one packet being lost this was at frame 11. 44
5.7 Histogram over RTT frames transmitted over Ethernet, sample size was 200 frame pairs. 45
5.8 RTT for pair of frames, sample size was 200 frame pairs transmitted over Ethernet 46

6.1 This figure illustrates a theoretical testbed setup for measuring the real-time event-to-memory latency of a LiDAR sensor. The setup uses a moving object that is positioned in front of the LiDAR. An additional reference sensor is used to trigger the timing measurement when the object reaches the defined position in front of the LiDAR. 54
6.2 This figure illustrates a theoretical testbed for measuring the real-time event-to-memory latency of an IMU. The setup uses a triggerable push device that applies a mechanical impulse to the IMU. When activated, the device sends a trigger signal and causes the IMU to move. The Raspberry Pi monitors the incoming IMU data until motion is detected. The latency is then calculated as the time difference between the trigger signal and the detected motion. 56

List of Tables

2.1 Relevant works for latency and package loss benchmarks . . . 15

5.1 LiDAR latency statistics for 20 trigger signals. The listed time sections represent different parts of the trigger-to-response pipeline. For all measurement points, the standard deviation was 0.008 ms. The main contributor to the measured variation was the LiDAR response time, since it was included in all measured sections of the pipeline. 36
5.2 IMU latency statistics for 20 samples. The confidence interval is the 95% CI of the mean. 36
5.3 Summary of measured IR camera latency sections, for 200 frames. 37
5.4 Summary of latency statistics pre-ISP for the Photon-to-frame-in-RAN experiement. The frame resolution was 1536 × 864, measured frame rate around 116 Hz. 39
5.5 A summary of the network transmission test is presented here. In total, 200 frame pairs were transmitted, corresponding to 3600 RGB packets and 400 IR packets. During the test, one packet was lost. 41
5.6 The network transmission test consisted of 200 transmitted frame pairs, corresponding to 3600 RGB packets and 400 IR packets in total. To enable comparison and evaluate the effect of packet loss, the frame pair affected by packet loss was excluded from the analysis and replaced with another valid frame pair from the recorded samples. As a result, the statistical analysis was performed using 200 valid frame pairs. 41
5.7 Summary of the network transmission test. In total, 200 frame pairs were sent, corresponding to 3600 RGB packets and 400 IR packets. Transmitted over Ethernet 43

List of acronyms and abbreviations

5G HAT	5G Communication Module
ACK	Acknowledgment
CoAP	Constrained Application Protocol
CSI	Camera Serial Interface
DMA	Direct Memory Access
GDPR	General Data Protection Regulation
GPIO	General Purpose Input/Output
I^2C	Inter-Integrated Circuit
IDE	Integrated Development Environment
IMU	Inertial Measurement Unit
INT	Interrupt
IoT	Internet of Things
IR	Infra Red
ISP	Image Signaling Processing
LED	Light-Emitting Diode
LiDAR	Light Detection and Ranging
LoS	Line of Sight
MQTT	Message Queuing Telemetry Transport
NEXSoS	Network-Enabled eXchange of critical information in Systems-of-Systems
NLoS	Non Line of Sight
QoE	Quality of Experience
QoS	Quality of Service
RAM	Random Access Memory
RGB	Red Green Blue
RTT	Round-Trip Time

TCP	Transmission Control Protocol
UART	Universal Asynchronous Receiver/Transmitter
UDP	User Datagram Protocol
UE	User Equipment
USB	Universal Serial Bus
UVC	USB Video Class
XR	Extended Reality

Chapter 1

Introduction

Modern emergency response operations, such as wildfire suppression, search-and-rescue missions, and disaster relief, depend on timely and accurate situational awareness for safe and effective decision-making. Effective decision-making requires the ability to process, collect, and share information across multiple systems and actors. Achieving a shared operational picture is challenged by fragmented data sources, heterogeneous systems, and constraints in communication infrastructure.

Advances in technologies such as cellular-networks and Extended Reality (XR) [1] will enable the possibility of improved situational awareness in emergency response situations. XR User Equipment (UE) will be connected via cellular-networks which allows XR UE to receive and transmit large amounts of data with low latency [1]. This enables real-time visualization and the possibility of coordinating the users carrying the XR devices and getting an overview of the users situation. Improving the overall situation and assist decision-making in an emergency response situation [2].

Despite these advancements, challenges still remain. Constraints in computation, energy consumption, and network reliability must be balanced against the need for low latency and high data fidelity. In this context, a central challenge is to build an overview and quantify the end-to-end latency between an XR user device and a remote system. The overview should be decomposed to get better insight into the latency introduced at different stages of data acquisition and transmission.

The increasing capabilities of modern sensors and embedded systems have enabled real-time monitoring and perception systems in applications such as robotics, autonomous vehicles, and XR. A typical configuration involves an integrated sensor node that collects heterogeneous data and transmits parts

of the data to remote servers for storage, processing, or visualization, since embedded systems often have limited processing power, memory, and energy consumption. However, real-time transmission of multi-sensor data presents significant challenges. Different sensing modalities, such as Light Detection and Ranging (LiDAR), cameras, and Inertial Measurement Unit (IMU), generate data with varying rates, formats, and requirements. When transmitted over mobile networks, the achievable data rate can fluctuate due to changing network conditions, making it difficult to maintain consistent performance. In such systems, end-to-end latency becomes a critical factor. The total delay is influenced by multiple stages, including data acquisition, processing, encoding, and network transmission. Variations in network conditions can further impact both latency and data delivery reliability, potentially degrading system performance in time-sensitive applications.

This thesis focuses on the design and evaluation of a multi-sensor system capable of transmitting heterogeneous sensor data over a 5G network. In particular, the work investigates end-to-end latency from the sensor node to a remote server, aiming to better understand how system components and network conditions contribute to overall delay. In the thesis, it is shown that the sensor node can transmit sensor data over a cellular network. System performance in terms of end-to-end latency was decomposed where it was feasible. The results show that the camera sensors and wireless transmission had the largest impact on the total transmission. However, latency contribution from the wireless transmission and the Infra Red (IR) camera should be further decomposed to get a clearer and more accurate insight into their respective latency contribution.

1.1 Background

In real-world operating environments, bandwidth variations are common especially in connection to mobile devices [3]. These variations directly affect the achievable latency, throughput, and data fidelity in real-time streaming. To cope with this, the system must dynamically adapt its data handling, for example by reducing frame rate or resolution for cameras, selectively thinning LiDAR point clouds, or adjusting IMU sampling frequency, depending on current network conditions.

Latency and packet loss are key network performance metrics considered in this project. Latency refers to the time it takes for data to travel from a source to a destination across a network. Packet loss, on the other hand, describes the proportion of data packets that are transmitted but fail to reach their intended

destination.

Both latency and packet loss have a significant impact on the quality of transmitted data. In particular, for real-time applications such as video streaming, high latency can introduce delays and buffering, while packet loss can result in missing or corrupted visual information. As a result, both factors play a crucial role in determining the overall performance and user experience of networked video systems [4], [5].

1.2 Purpose

The project is developed within the umbrella of the Network-Enabled eXchange of critical information in Systems-of-Systems (NEXSoS) project, where the primary focus is on developing a system which could support firefighters and emergency response teams in forest fire management by providing a clearer situational awareness [2]. The main purpose of this project is to conduct experiments with the developed sensor node. Experiments that result in the quantification of the decomposed end-to-end latency between the sensor node and the remote system. To better understand system behavior and identify performance bottlenecks. As previously mentioned the project is within an ongoing project. Therefore, the main user group consists of fire service personnel. However, the underlying technology is not limited to this application and can be extended to a wide range of use cases, including autonomous systems, remote monitoring, and industrial or research contexts. The primary ethical consideration arises from the use of cameras, which may capture identifiable individuals or sensitive environments; this aspect will be addressed through privacy-conscious design and discussion of mitigation measures. The technology and findings presented in this thesis are intended to benefit anyone seeking to implement real-time data streaming solutions. Ericsson, as the collaborative partner for this work, stands to benefit most directly, with the results informing and supporting their future projects and developments in this area [2].

1.3 Problem and Definition

The problem revolves around decomposing, and analyzing the total latency between the sensor node developed and the receiving server. Specifically, how much can the latency be decomposed, is it possible to measure the latency for a certain module given the resources we have, and if the latency can be

measured, what method should be applied to measure it?

The scientific experience expected to be gained is a clearer understanding where latency bottlenecks occur between an XR prototype device and server. The engineering experience expected to be gained is from the testbeds developed to measure latency for identified modules.

The primary scientific challenge is understanding how different latency components contribute to perceived XR responsiveness and user experience, while the engineering challenge lies in accurately measuring these latency sources.

1.4 Goals

The Goal of the project can be divided into the two following sub-goals:

1. Deliver a validated, reusable sensor node prototype capable of real-time multi-sensor streaming over a mobile network.

   Success criteria: Working prototype that is capable of streaming LiDAR, IMU, IRand Red Green Blue (RGB) data to a cloud server. Prototype should also be able to handle latency benchmark.

2. Analyze, decompose, and quantify the end-to-end latency between the sensor node and a remote server receiving the transmitted data.

   Success criteria: The latency contribution of different stages in data handling and transmission is identified and analyzed. The results are reproducible across multiple experimental runs.

1.5 Research Methodology

The research methodology adopted in this thesis is based on a design science approach, where a sensor node is designed and implemented. This is complemented by an experimental methodology used to evaluate system performance through experiments and quantitative measurements based on the sensor node at its network environment.

1.6 Delimitations

This thesis is focused around developing a 5G-connected sensor node and to conduct latency based experiments with the sensor node. The scope excludes

additional sensor types and multi-node or fleet configurations. Exclusive latency measurement points that are outside of the scope for this project include latency from the 5G-modem, latency within the hardware of the cellular network, any latency in regards to computation of sensor data for an XR application.

1.7 Structure of the Thesis

Chapter 2 presents the reader with relevant information about the project domain, the challenge within the domain and what is to be expected in the results of the report. Chapter 3 describes the methodology used to address the research question, including research strategy, experimental design and quality assurance. Chapter 4 documents the implementation and engineering work, including hardware and software architecture, sensor integration, and adaptation mechanisms, latency measurement methods. Chapter 5 presents the experimental results and analysis. Chapter 6 discusses implications, limitations, and ethical considerations based on the results and implementations. Chapter 7 contains future work that can be carried out within the scope of the project and concluding remarks.

Chapter 2

Background

This chapter presents the relevant information needed to understand both the methodology and the results of the thesis. The purpose is to provide the necessary context for the measurement approach as well as terms used in the result. The chapter also discloses related work that serves as a foundation for the thesis.

2.1 Internet of Things

Internet of Things (IoT) is a term first introduced in 1985, referring to systems capable of connecting to the internet to transmit information and receive instructions [6]. Since its introduction, the concept has continued to evolve, and IoT devices are today deployed across a wide range of domains, including industrial automation, healthcare, smart infrastructure, agriculture, transportation, energy management, and consumer applications. In industry, IoT is used to connect machines, sensors, and control systems, enabling real-time monitoring, predictive maintenance, and improved process efficiency. In healthcare, connected devices can support remote patient monitoring, medical asset tracking, and faster access to patient data.

In smart cities and infrastructure, IoT technologies are applied in areas such as traffic management, environmental monitoring, smart lighting, and waste management [7]. Agriculture also benefits from IoT through precision farming, where sensors are used to monitor soil conditions, weather, crop health, and livestock [8]. These applications demonstrate that IoT is a broad technological paradigm in which physical objects are equipped with sensing, communication, and computational capabilities to support data-driven decision-making across many sectors.

2.1.1 Sensor Node

Sensor nodes are a category of IoT devices in which multiple sensors are integrated onto a single platform, such as a Raspberry Pi, enabling the collection and transmission of data to a remote server or processing unit [9]. Sensor nodes are commonly used in applications such as environmental monitoring, industrial automation, and smart infrastructure, where continuous data collection is required [10]. The design of a sensor node typically involves considerations such as power consumption, processing capability, and communication protocol, which are chosen based on the requirements of the target application [11].

2.1.2 Edge and Cloud Processing

Data processing is a fundamental component of IoT system architecture. Two predominant approaches exist for handling data in such systems, edge processing and cloud processing [12].

Edge processing refers to the execution of data processing tasks directly on the device itself. This approach reduces transmission overhead and can offer lower latency, making it suitable for time-sensitive applications [12], [13]. However, the processing capability is inherently constrained by the hardware of the device, which may become a limiting factor when the computational demands of the application are high [13].

Cloud processing addresses this limitation by offloading data to remote servers, where significantly greater computational resources can be allocated [13]. This makes cloud processing well suited for data-intensive tasks. The trade-off, however, is the additional latency introduced by transmitting data to and from the cloud, which may not be acceptable in all use cases [12], [13].

The choice between the two approaches depends on the specific requirements of the application, including processing demand, latency tolerance, and available resources. The system developed in this project adopts a cloud processing approach, since the NEXSoS project has a vision of a system that processes large volumes of visual data. That computation would need to be offloaded to the cloud.

2.2 Light Detection and Ranging

LiDAR, or Light Detection and Ranging, is a sensing technology used to measure the distance between the sensor and surrounding objects. It operates

by emitting light pulses and measuring the time it takes for the emitted light to travel to an object and return to the sensor. This round-trip time can then be used to calculate the distance to the object [14].

In addition to distance measurement, the strength of the returned light can provide information about the confidence or quality of the measurement. A stronger returned signal generally indicates that more of the emitted light was reflected back to the sensor, while a weaker returned signal may indicate lower measurement certainty. This can depend on factors such as the distance to the object, the angle of reflection, and the surface properties of the object being measured.

2.3 Inertial Measurement Unit

An IMU, or Inertial Measurement Unit, is a sensor unit used to measure motion and orientation-related quantities. It commonly consists of accelerometers and gyroscopes, and in some cases also includes a magnetometer. The accelerometer measures linear acceleration, while the gyroscope measures angular velocity. Together, these measurements can be used to estimate movement, rotation, and orientation of the sensor [15].

In this thesis, the IMU is relevant because its measurements contribute to understanding the timing behaviour of sensor data in the system. Since IMU data is often sampled and transmitted at high rates, the latency introduced during data acquisition and communication can affect how quickly motion information becomes available for processing. Understanding the basic function of an IMU therefore provides useful context for the latency measurements and analysis presented later in the thesis.

2.4 Overview of Latency

Latency refers to the time delay between the transmission of data from a source and its reception at a destination. It is a critical performance metric in IoT systems, particularly in applications that require real-time or near real-time responsiveness, such as industrial automation, healthcare monitoring, and autonomous systems [4], [16]. In such systems, excessive latency can degrade performance, reduce reliability, or even lead to system failure.

Latency in an IoT system can be broadly decomposed into contributing components [4]:

- **Application latency:** This refers to delays introduced by the software running on the device, including data processing, sensor sampling, and task scheduling. Inefficient algorithms or limited processing resources can significantly increase application introduced latency.
- **Link (communication) latency:** Is the time it takes for a data packet to travel from its source to its destination. The link latency sum can be decomposed into four components. Propagation, transmission, processing, and queuing delay.
  - **Propagation delay:** The time it takes for a signal to travel through a wired or wireless medium. The two factors that influence propagation delay is the type of medium and distance.
  - **Transmission delay:** The time it takes to push all bits of a package into the communication link. The delay is determined by the available bandwidth on the link and the size of the data package.
  - **Processing delay:** The time spent by routers or switches processing the packet header and determining the forwarding destination.
  - **Queuing delay:** The time a package spends in a buffer of a device before it is transmitted. This delay depends on the current network traffic and congestion.
- **Hardware latency:** This refers to delays introduced by physical hardware components, including sensors, microcontrollers, communication interfaces, and peripheral devices. Hardware limitations such as clock speed, bus contention, and interface delays can impact overall system latency.

The total end-to-end latency is the cumulative effect of these components, and its relative importance depends on the specific application requirements. For instance, low-latency communication is essential in time-critical systems, whereas higher latency may be acceptable in applications such as environmental monitoring. Consequently, understanding and optimizing each component of latency is essential for the design of efficient and reliable IoT systems [16].

2.5 Device Level Communication Protocols

Device-level communication is a fundamental design consideration in IoT systems, governing how sensors and peripheral components exchange

data with a central processing unit. Unlike network-layer protocols designed for internet-facing transmission, device-level protocols operate over comparatively short distances, typically within a single printed circuit board or between directly connected components [17]. The selection of an appropriate protocol is not trivial and depends on several system-specific factors, including the number of connected devices, required data throughput, physical distance between components, and available hardware interfaces [18]. For this project, the protocols of primary relevance are Inter-Integrated Circuit (I^2C), Universal Asynchronous Receiver/Transmitter (UART), Universal Serial Bus (USB) and Camera Serial Interface (CSI), each of which offers distinct characteristics suited to different communication requirements.

2.5.1 USB Video Class

USB Video Class (UVC) is a standardized USB device class for video streaming devices. It defines how a camera exposes video formats, frame sizes, frame intervals, control interfaces, and video payloads over USB. The purpose of the standard is to allow video devices to be used through a common driver model rather than requiring a device-specific driver for each camera implementation [19].

In this work, the thermal camera stream is transferred using UVC. The camera board packetizes each completed thermal frame and transmits it over USB as a video frame. On the Raspberry Pi, the frame is received through the Linux UVC pipeline and then read by the measurement software. Therefore, the measured UVC delivery delay represents the time from the board-side USB transmission point to the point where the frame is available to the application on the Raspberry Pi. This includes USB transfer, driver handling, buffering in the video pipeline, and memory frame reception. It should not be interpreted as only the electrical propagation time on the USB cable.

2.6 Network Transmission Protocols

A fundamental consideration when designing IoT systems is the ability to transmit data in real-time. While the collection of sensor data is an essential first step, the efficiency of data transmission ultimately determines the practical usefulness of the system.

Data transmission in IoT systems is typically handled over the internet using one of two foundational transport protocols: Transmission Control Protocol (TCP) or User Datagram Protocol (UDP) [20]. TCP provides

reliable, ordered delivery of data through acknowledgment mechanisms, making it well suited for applications where data integrity is critical. UDP, in contrast, offers lower overhead and reduced latency by omitting these guarantees, which makes it preferable in time-sensitive applications where occasional data loss is acceptable [20], [21]. Built on top of these transport protocols are application-layer messaging protocols, such as Message Queuing Telemetry Transport (MQTT) and Constrained Application Protocol (CoAP), which define how data is formatted, routed, and processed [20]. The choice of protocol at each layer depends on the specific requirements of the system, including network conditions, device capabilities, and the nature of the data being transmitted. In this project only UDP and CoAP will be used.

2.6.1 Transmission Control Protocol

TCP is a connection-oriented transport protocol that provides reliable, ordered, and error-checked delivery of data between devices [22]. Before any data is exchanged, a connection between the sender and receiver is established through a process known as the three-way handshake, ensuring that both parties are ready to communicate prior to transmission [23]. The reliability guarantees offered by TCP make it well suited for IoT applications where data integrity is critical, such as the transmission of configuration updates, commands, or data that cannot be lost or arrive out of order [24]. However, the overhead introduced by connection establishment, acknowledgments, and retransmission of lost packets can increase latency, which may be a drawback in time-sensitive or resource-constrained environments [24].

2.6.2 User Datagram Protocol

UDP is a connectionless transport protocol that prioritises speed and low overhead over reliability [21]. Unlike TCP, UDP does not establish a connection before sending data, nor does it guarantee delivery, ordering, or error recovery. Packets may arrive out of order, be duplicated, or be lost entirely without any notification to the sender [21].

Despite these characteristics, UDP is widely used in applications where low latency is more important than -guaranteed delivery [20]. Typical use cases include the continuous streaming of sensor data, audio, and video feeds, where receiving the most recent data promptly is preferable to waiting for retransmission of lost packets [20]. The reduced overhead also makes UDP a practical choice for resource-constrained devices with limited processing

power and energy capacity [21].

2.6.3 Constrained Application Protocol

CoAP is a lightweight messaging protocol designed for use with constrained devices, such as low-power sensors and embedded systems with limited processing capacity [25]. It is built on top of UDP, which means it does not require a connection to be established before data is transmitted. Unlike TCP, which performs a handshake between sender and receiver before any data exchange can begin, UDP sends packets directly to the destination without waiting for acknowledgement of a connection. This makes CoAP particularly well suited for environments where network conditions are unreliable or bandwidth is limited, as the overhead of maintaining a connection is avoided entirely [26]. CoAP does however include its own lightweight reliability mechanisms, such as confirmable messages that request an acknowledgement from the receiver, allowing developers to choose between reliable and non-reliable delivery depending on the use case. This balance between simplicity and optional reliability makes CoAP a practical choice for IoT applications where devices must operate efficiently under constrained conditions.

2.7 Fifth Generation of Cellular Network

5G is the fifth generation of cellular-networks. It outperforms the fourth generation of cellular-networks in terms of latency, download speeds, and positioning accuracy [27]. A cellular-network is a system that allows for wireless communication between mobile devices and the internet via radio waves.

2.8 Packet Loss

Packet loss is an important factor when transmitting data over a network, as lost packets result in incomplete or degraded information at the receiver. In real-time applications such as video streaming, packet loss is particularly critical due to the use of UDP, which does not provide retransmission mechanisms for lost data. Unlike reliable transport protocols, UDP prioritizes low latency over reliability, making it suitable for time-sensitive applications but more

susceptible to quality degradation. Consequently, packet loss in video streams can lead to visual artifacts, reduced resolution, and interruptions in playback quality [5]. Understanding and measuring packet loss is therefore essential when evaluating the performance of networked video transmission systems.

2.9 Bandwidth

Bandwidth represents the maximum rate at which data can be transmitted over a network and is a critical factor in real-time systems. Limited or fluctuating bandwidth can lead to increased latency, packet loss, and degraded system performance.

Bandwidth in wireless and mobile networks is variable due to changing conditions in the networks. Influential factors on the network conditions are user mobility, signal interference, and varying network load. Which causes fluctuations in achievable data rate from a wireless network. These rapid fluctuations in wireless communications and streaming systems are well documented, where maintaining a stable data rate to preserve Quality of Service (QoS) is challenging [28].

If the data amount generated exceeds the current available network bandwidth, data needs to be accumulated in buffers, which will lead to an increased queuing delay. Which in a real-time system will result in an increased end-to-end latency, and it may also result in data package loss. If the accumulation of data in the buffers cause the buffers to overflow. These effects then cause a decrease in the overall QoS which also negatively impacts the Quality of Experience (QoE) for the end-user [29].

2.10 Kubernetes

Kubernetes is an open-source platform for managing containerized applications. It is commonly used to automate deployment, scaling, and management of containers [30]. A Kubernetes cluster consists of a control plane and one or more worker nodes. The control plane manages the cluster, while the worker nodes run the application workloads [31].

Applications in Kubernetes are usually run inside pods, which are the smallest deployable units in Kubernetes. A pod can contain one or more containers that share network and storage resources [32]. Since pods can be restarted or moved, Kubernetes services are used to provide a stable network endpoint and forward traffic to the correct pod [33].

In this project, Kubernetes was used to run the receiving server inside a container. A Kubernetes service was configured so that incoming data could be routed to the correct container.

2.11 Related Work

In this section, relevant existing work is reviewed. These works provide the theoretical and technical foundation for this project and some related work is referenced throughout the development and analysis. The referenced benchmarks for packet loss and latency are detailed in Table 2.1.

2.11.1 Latency in Video Transmission

The system developed in this project handles multiple sensor types, with video streaming being the central component of the use case. Due to the continuous and real-time nature of video data, it is treated as the primary bottleneck in end-to-end transmission for the purposes of this evaluation. For this reason, existing research on video streaming latency serves as a relevant benchmark for assessing the overall performance of the system.

One study focused on the QoE in first-person gaming scenarios, where the participant's perspective is directly tied to real-time visual feedback, a condition closely comparable to the use case of this project. The study found that latency must remain below 80 ms to maintain an acceptable QoE. Exceeding this threshold was shown to noticeably degrade the user experience, suggesting that 80 ms can be treated as an upper bound for tolerable latency in real-time, first-person applications [34].

A second study examined video streaming under varying conditions, among which an adaptive resolution stream used in a sports broadcasting context achieved a measured latency of 22.5 ms. This demonstrates what is achievable with optimized, adaptive streaming pipelines and can be considered a reference point for low-latency performance [35].

Since video is the component most central to the project's use case, these latency values are used as the primary benchmarks when evaluating the system's performance.

2.11.2 Packet Loss in Video Transmission

Packet loss is an inherent phenomenon in network-based data transmission and constitutes a critical quality metric in the context of real-time media

Table 2.1: Relevant works for latency and package loss benchmarks

Reference	Application Context	Key Findings	Relevance to This Project
[34]	First-person gaming (real-time video)	Latency must remain below 80 ms to maintain acceptable QoE. Higher latency significantly degrades user experience.	Defines an upper bound for acceptable latency in real-time video applications.
[35]	Adaptive video streaming (sports broadcasting)	Achieved latency of 22.5 ms using optimized adaptive streaming techniques.	Represents a lower-bound performance target for low-latency systems.
[5]	Packet loss impact on QoE	Low packet loss rates (0%–0.1%) result in no perceptible degradation in video quality. At moderate loss rates (0.3%–1%), user perception becomes inconsistent, indicating a transition region where QoE begins to degrade.	Provides empirical packet-loss thresholds for defining the benchmark's target range and assessing the expected impact of packet loss on perceived video quality.

streaming. The perceptual impact of packet loss on end-user experience is closely tied to the magnitude of the loss rate. Empirical findings indicate that at low packet loss rates, specifically within the range of 0% to 0.1%, viewers demonstrate no perceptible degradation in perceived stream quality, suggesting that the human visual system is largely insensitive to loss events at this threshold. However, as packet loss rates increase to the range of 0.3% to 1%, user perception becomes notably inconsistent, with viewers reporting divergent assessments of stream quality, a pattern indicative of a transitional zone in which QoE begins to deteriorate [5]. These findings underscore the importance of maintaining stringent network performance standards in streaming applications. Accordingly, this project establishes a target packet loss rate of 0% to 0.1% as the operational benchmark, with the aim of ensuring a consistently high and uninterrupted viewing experience for end users.

2.11.3 5G Networks for XR Use Cases

Related work has explored the integration of 5G networks with XR applications. In a study, two different testbeds for 5G and XR integration were implemented [36]. A private 5G for an indoor XR-assisted facility tour, and a public 5G for XR-assisted medical emergency care. Latency and packet loss performance of the XR use cases were evaluated.

The private 5G testbed decomposed latency into eight stages, covering

encoding, uplink and downlink transmission, decoding, pre-processing, inference, post processing, and information overlay. Results show that communication latency scales with the frame resolution. Inference latency contributed the most to the overall latency according to the results. These results however are ambiguous, since inference time decreases as frame resolution increases. Without any comment from the authors to explain this outcome. While decoding, pre-processing, and post processing latencies where relatively small and consistent. The results also concluded that UDP provided a more deterministic throughput in both Line of Sight (LoS) and Non Line of Sight (NLoS) than TCP. While TCP provided higher bandwidth in LoS but degraded in NLoS.

The public 5G testbed decomposed the latency of the encoding and transfer pipeline the same as in the private 5G testbed. Increase in inference latency had a closer relation to frame resolution increase in this testbed. 330 x 224 had the least inference time of 52.16 ms. Decoding, pre-processing, and post processing had a relative small impact on overall latency. Communication latency increased with frame resolution, the overall average latency was higher than the private 5G testbed. UE satisfaction results in the form of the percentage of packages a XR UE receives was measured. The results showed that TCP in LoS mode and UDP in LoS and NLoS had 99% rate or higher of received packages.

This thesis-project will examine the decomposed latency in an XR-sensor transmission pipeline, similar to what has been done in this study [36]. The results from this study which compare the UE satisfaction of the two protocols UDP and TCP can assist us in choosing a suitable protocol for our testbed given that our testbed will be NLoS. Parts of this study are outside the scope of our thesis-project. Our project will not implement any object detection and will therefore not decompose any latency related to inference. The study also compares two different 5G testbeds with specified network parameters. While our testbed will utilize a single stand-alone 5G lab network.

2.11.4 Motion-To-Photon Tolerance Latency

Motion-to-photon latency is an inevitable delay in XR. It is the time between the user's physical movement and the user seeing the visual feedback on the XR device. Quantifying this delay and identifying the perceptual tolerance of a user is crucial to ensure a positive user experience. A study has examined the perceptual tolerance to motion-to-photon latency. More specifically, it investigated the threshold for the perception of delay and the user acceptance

of the delay exceeding the threshold [37]. The study brings up how different head motion events, such as Motion-Static-Alternate, an acceleration or deceleration in one direction, or Motion-To-Reverse, a movement reverses direction. Impacts the perception of the Motion-to-photon latency. According to the study factors like rotation angle and angular velocity in the motion events also influence the motion-to-photon perception [37]. The experimental results show that the test subjects were more intolerant of photon-to-motion latency in Motion-To-Reverse than Motion-Static-Alternate. The latency perception threshold is approximately 23 ms according to the results of the study [37]. It is also stated in the results that when the angular velocity drops the perception threshold increases. Where the maximum threshold is approximately 41 ms with an angular speed of 20 °/s.

Although the work of this project focuses on latency contribution within a sensor node server system rather than motion-to-photon latency. The findings of the study provide valuable context for interpreting the latency results from our experiments. The perception threshold from the study and user acceptance levels provide insight into the delay ranges that are considered acceptable in XR systems.

Chapter 3

Methods

The engineering-related content of this thesis project was divided into two main parts. The first part concerned the design and implementation of a sensor node capable of capturing and transmitting multi-sensor data over a 5G network. The second part focused on conducting experiments on the sensor node to gain insight into performance-related metrics, such as decomposed latency.

The research methodology adopted in this thesis was based on a design science approach. A multi-sensor edge system was designed and implemented, followed by an experimental evaluation in a test environment. This enabled a quantitative analysis of the system's latency and data transmission behavior.

A purely simulation-based approach was also considered as an alternative research methodology. This approach was rejected because a simulated environment would have relied on abstract models, which may not have accurately captured real-world behavior related to hardware constraints and varying network conditions. Consequently, such an approach could have produced results that were less representative of the final system's actual performance.

Qualitative research was also considered. However, this methodology was deemed unsuitable for the scope of this thesis. Since the methodology is mainly focused around meanings, experiences, interpretations, and context rather than to produce precise numerical performance. This work is focused on experimental evaluation of latency in a hardware-based sensing system, hence a quantitative approach was deemed more appropriate.

3.1 Research Process

The research process for this thesis-project followed the following framework.

1. Problem identification: We defined the problems around development of the sensor node, latency breakdown, and latency measurements the project encountered.

2. Review existing literature: Existing information on topics such as latency, variable bit-rate transmission, edge computing, transport protocols and application-layer messaging protocols was reviewed to create a theoretical foundation for the thesis-project.

3. System design: We designed the hardware architecture of the sensor node, which included selection of the computing platform, sensors, communication interfaces, and transmission solution for data delivery.

4. System implementation: Based on the design choices made we started development of a functional Raspberry Pi based edge sensor node prototype. With the capability of multi-sensor data transmission via 5G.

5. Data collection: Decision were made on the feasibility of measuring the different latency modules in the total end-to-end latency. We then conducted measurements to receive performance metrics such as latency and packet loss.

6. Data analysis: We analyzed the collected data to evaluate the latency impact of specific modules.

7. Conclusion: A conclusions was drawn based on the experimental results and the results were related to the research question.

3.2 Data Collection

The sensor node developed in this thesis collects several types of measurements to support environmental perception and later analysis. These measurements include inertial data from the IMU, distance and spatial information from the LiDAR sensor, visual data from the RGB camera, and thermal/infrared information from the IR camera. Together, these sensing

modalities provide complementary information about the surroundings, allowing the sensor node to capture both motion-related data and environmental features. All collected data is logged for subsequent analysis, with access restricted to the research team to ensure security and integrity throughout the process. The deployment of a sensor node with integrated camera technology raises important social considerations. Cameras, even when used for research purposes, have the potential to capture individuals without their consent and would be a breach of their privacy.

The use of camera-equipped sensors introduces a ethical responsibilities that must be carefully managed. A central consideration is informed consent. Where the sensor node operates in environments where identifiable individuals may be captured, steps must be taken to ensure that those affected are made aware of the data collection activity and its purpose. In cases where obtaining direct consent from every individual is not feasible, appropriate safeguards must be implemented to minimize the risk of personal identification.

Anonymization of data is therefore prioritized wherever possible. Visual data that is not essential to the research objectives should not be retained, and data storage should be limited to the minimum period necessary for analysis. This aligns with the principles outlined in the General Data Protection Regulation (GDPR), which establishes legal obligations regarding the collection, storage, and processing of personal data, particularly data that could be used to identify individuals.

Furthermore, the data collected through the sensor node is used exclusively to serve the stated research purpose and is not shared with external parties without clear justification. The research is conducted in a manner that allows any concerns raised by affected individuals to be acknowledged and addressed, reflecting a commitment to ethical integrity throughout the study.

3.3 Experimental design and Planned Measurements

The benchmarks were conducted in a controlled experimental setup. The Raspberry Pi-based sensor node was connected to a cellular network via a 5G modem, enabling wireless data transmission. Sensor data generated on the device was transmitted to a server deployed within the cloud.

This architecture allowed for improved control over the server-side environment while maintaining representative network behavior. Furthermore, it could assist measurement and analysis of end-to-end latency, as well as the

decomposition of latency across different segments of the communication pipeline (e.g., device processing, network transmission, and server-side handling).

3.3.1 Experimental Setup

To replicate the experimental setup, the following hardware components are required: a Raspberry Pi 4 Model B, an IMU, a LiDAR sensor, an RGB camera, and an IR camera. Additionally, a 5G Communication Module (5G HAT) with an active SIM card is necessary to enable connectivity to a cellular network and a Digital Discovery from DIGILENT.

On the server side, it is recommended to use a self-managed (non–third-party) server infrastructure. This provides greater control over the execution environment and facilitates more accurate measurement and analysis of system performance, particularly latency. To fully replicate the testbed, the server should be deployed within the cloud, ensuring consistency with the original experimental conditions and enabling scalable and reproducible orchestration of services.

3.3.2 Hardware and Software to be Used

Notable hardware requirements include a single-board computer, such as a Raspberry Pi 4 model B, a 5G modem capable of connecting to the single-board computer for network connectivity, an IMU, a LiDAR sensor, an RGB camera, an IR camera and a logic analyzer.

Software requirements are minimal. A code editor or Integrated Development Environment (IDE) of choice is sufficient for development, along with a server setup for data reception, processing, and analysis. As well the tests that were performed in this thesis used the programming language Python.

3.3.3 Expected Phase Delay under Asynchronous Sampling

When a Light-Emitting Diode (LED) or event is triggered asynchronously with respect to a camera frame cycle, the activation time is uniformly distributed within a single frame period. Let T_{frame} denote the frame period and t_{phase} the time offset between the LED event and the next frame capture. Because the LED trigger is not synchronized with the camera exposure clock, it can occur

at any instant within the frame interval. Thus impacting the overall camera sensor acquisition latency. This can be modeled as a uniform distribution:

$$t_{\text{phase}} \sim \mathcal{U}(0, T_{\text{frame}}) \tag{3.1}$$

The expected value of the phase offset is then given by the mean of the uniform distribution:

$$E[t_{\text{phase}}] = \frac{T_{\text{frame}}}{2} \tag{3.2}$$

Thus, over a large number of asynchronous LED activations, the phase offsets uniformly cover the interval $[0, T_{\text{frame}}]$, and the average converges to the midpoint of the frame period.

3.3.4 Sample Size

In this thesis, different sample sizes were used in the experiments. The main focus and primary interest from Ericsson were to investigate as much as possible of the camera feeds and the network performance when transmitting video frames over the 5G network. The LiDAR and IMU were also of interest, but they were not studied as extensively. A more in-depth discussion of the effect of sample size on the results is presented in Section 6.1.

3.4 Assessing Reliability and Validity of the Data Collected

Ensuring the reliability and validity of both the research method and the data collected is fundamental to establishing the credibility and trustworthiness of this study. In experimental research, reliability refers to the consistency and repeatability of results, while validity concerns the accuracy and appropriateness of what is being measured and how it is being measured.

3.4.1 Validity and Reliability of Method

The experimental methodology was selected because it enables controlled and systematic experiments to be run on the sensor node, allowing specific conditions to be defined, manipulated, and measured in a structured and repeatable manner. This approach directly aligns with the objective of

decomposing and quantifying latency in the system, making it a suitable choice for this type of technical investigation.

A reliable method is one that, if followed by another researcher under the same conditions, would yield comparable results. To support this, the testing procedure was documented in sufficient detail to allow replication, including the configuration of the sensor node, the streaming setup, the conditions under which tests were conducted, and the approach to data storage and logging.

The same procedure was applied consistently across all test sessions, using identical hardware configurations and software settings throughout. This consistency reduces the risk of procedural variation influencing the results and strengthens the dependability of the method. Where deviations from the planned procedure occurred, these were noted and accounted for in the analysis. By maintaining a structured and well-documented approach, the method provides a stable and repeatable basis for the experimental evaluation.

3.4.2 Validity and Reliability of Data

For the data to be considered valid, each sensor must produce output that genuinely corresponds to the phenomenon being observed, rather than reflecting noise, interference, or miscalibration. Prior to testing, sensors were verified to ensure they were operating within expected parameters. Where applicable, sensor readings were compared against reference values or known benchmarks to confirm that the measurements produced were accurate and meaningful. Camera output was assessed to determine whether the visual data captured provided an accurate representation of the monitored environment under varying lighting and spatial conditions.

Potential threats to data validity, including environmental interference, signal noise, and sensor drift. They were monitored throughout the testing process to ensure the validity of the produced data.

To assess data reliability, the sensor node was tested across several sessions under identical conditions, and the results were compared to identify patterns of consistency or divergence. Stable and repeatable readings across sessions were taken as indicators of reliable data. The real-time streaming nature of the system introduced additional considerations, as network latency, connection interruptions, and data loss that could affect the completeness and consistency of the received data. The incoming stream was monitored throughout testing to detect anomalies, gaps, or irregularities in the data flow.

Sessions in which significant data loss or notable instability was observed were documented, and the impact of these occurrences on the overall dataset

was considered during analysis. By conducting multiple test sessions and cross-checking results, a consistent baseline of performance was established, providing confidence in the reliability of the data collected and its suitability for the subsequent analysis.

3.5 Planned Data Analysis

The collected data were not taken at face value, but were instead evaluated to determine its relevance and to identify the influence of external variables that may have affected the measurements. The analysis focused on key performance metrics, such as latency and packet loss.

3.5.1 Data Analysis Technique

Descriptive statistics were used to get an understanding regarding trends regarding how the data was sent. Data visualization was also implemented to give a better understanding regarding performance and trends.

3.6 Evaluation Framework

The project used a benchmark framework, to be able to evaluate the latency performance of the developed system.

QoS were used as a comparison framework, which provided an initial comparison between metrics such as a latency and packet loss received. Which assisted in an initial evaluation of benchmark metrics.

Chapter 4

Implementation

This chapter describes the implementation of the sensor node and the testbeds used to evaluate the different sensor pipelines. The measured points within each pipeline are presented, together with the design choices made for the corresponding test setup. This includes how each sensor was integrated, how the measurement points were defined, and how the testbeds were constructed to enable latency measurements for the respective sensors.

4.1 Construction

The project began with the development of a testbed consisting of a sensor node and a cloud shown in Figure 4.1. The sensor node was implemented using a Raspberry Pi, where all sensors were physically connected and the required software dependencies were installed.

For latency measurements of the RGB camera, the testbed was extended with a dedicated measurement setup. The camera and an LED were mounted at opposite ends of an enclosed box to control the visual environment. The LED was connected to a General Purpose Input/Output (GPIO) pin on the Raspberry Pi, enabling precise timestamping of the activation signal used as a reference event.

The IMU and LiDAR sensors were integrated into the sensor node and connected via their respective communication interfaces.

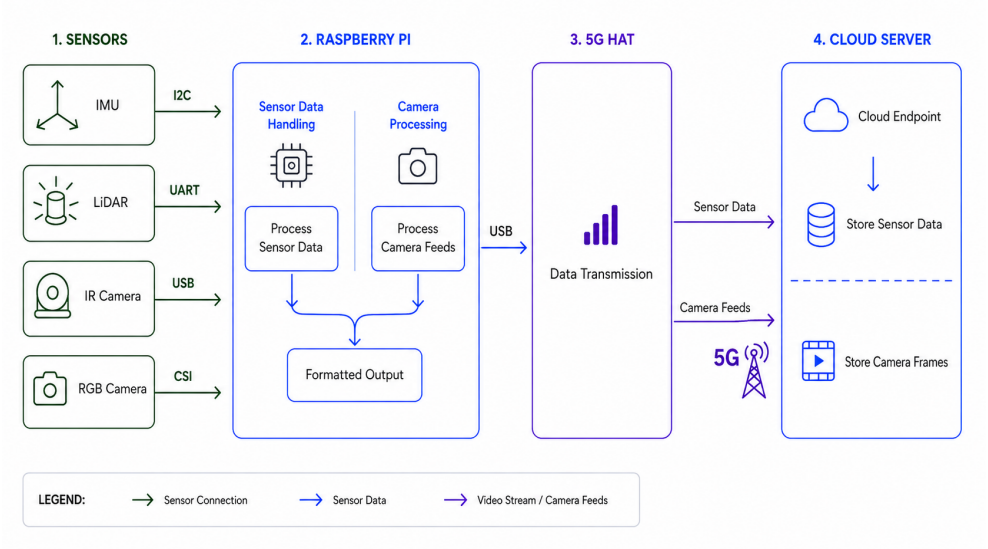


Figure 4.1: The picture shows the high level architecture design of the system. The four sensors are connected to the Raspberry Pi and handled differently depending on type. The 5G hat acts as the bridge between the device and the cloud.

4.2 Experiment Setup

To evaluate the timing characteristics of each sensor, individual experimental setups were defined. These setups include real-time event measurement testbeds, as well as measurement testbeds designed to assess internal latency at different stages of the processing pipeline. This is followed by a testbed for evaluating the network transmission latency of video frames.

4.2.1 RGB Camera

The purpose of this experiment was to capture the end-to-end latency between a real world optical event and the moment the corresponding raw camera data was available in Random Access Memory (RAM) prior Image Signaling Processing (ISP). The Raspberry Pi Camera Module 3 uses a IMX708 image sensor which operates in a rolling shutter mode [38]. The latency was measured by triggering the LED via a GPIO signal and capturing the resulting visual event with the camera. The timestamp of the LED activation was compared to the timestamp of the corresponding image frame in which the light became visible see Figure 4.2.

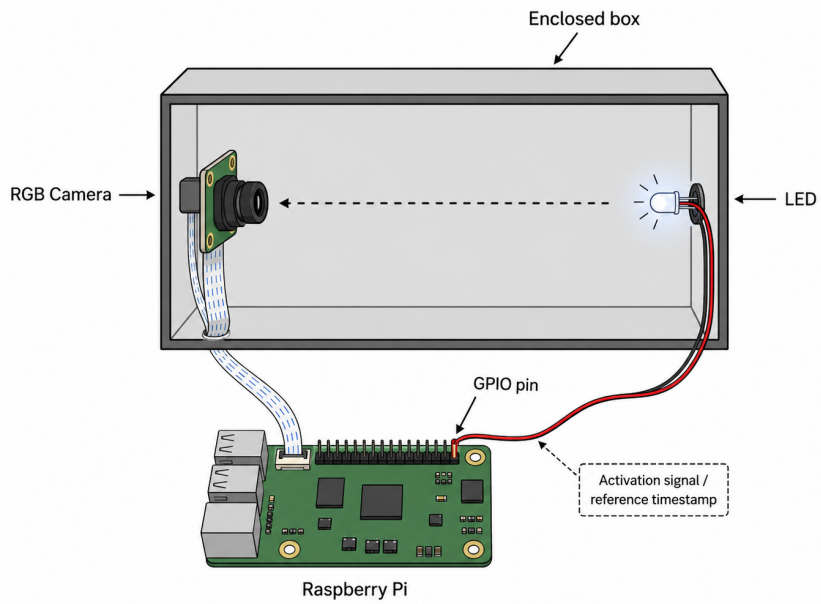


Figure 4.2: This picture represented the testbed implemented to test a real-time event until it reaches memory in the device. The Raspberry Pi turns on a LED and times the time it takes after the LED is turned on until the event has arrived in device memory.

The script that ran the benchmark handled two different timestamps when measuring latency. The first timestamp came from the camera sensor, which was the time when the sensor started exposure of a frame. The second timestamp came from when the LED was triggered via GPIO. The two timestamps rely on the same underlying clock source from the Raspberry Pi, but use different kernel clocks. The script handled this by calculating the offset to keep the timing measurements in the same domain. The script avoided comparing stales frames in buffer by comparing start-of-frame timestamp with LED timestamp. It also compared the mean value of the frame to determine if it exceeded the brightness threshold. If the value exceeded the brightness threshold, a timestamp sum of start-of-frame and frame duration was saved as the end-to-end latency for that trail.

The total latency captured in the experiment consists of several sequential stages within a rolling-shutter camera pipeline:

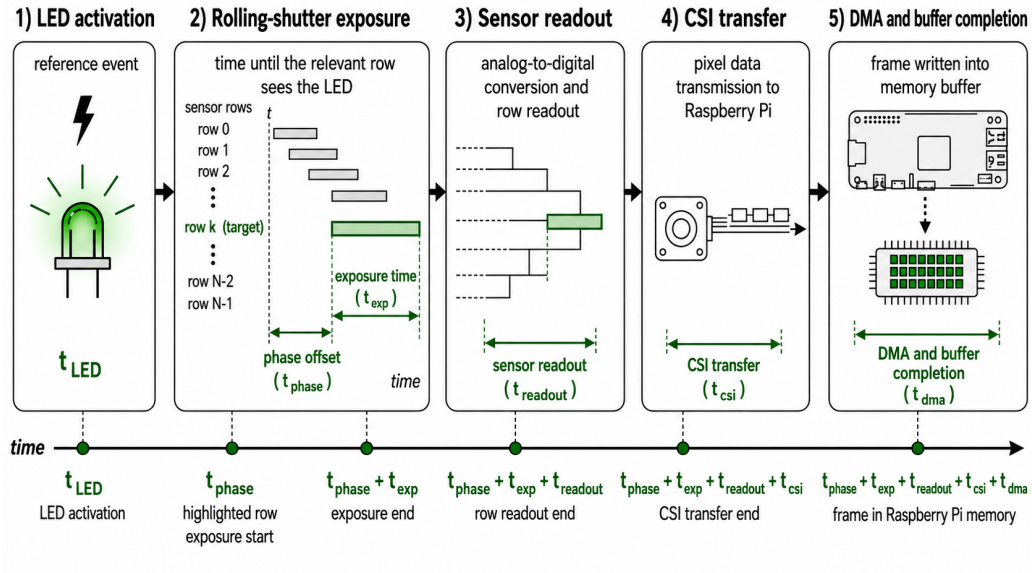


Figure 4.3: Figure shows the different stages that make up the total latency pipeline of photon-to-frame-in-RAM latency, for a camera sensor using rolling shutter. Latency contribution from stages such as t_{csi}, t_{dma} and t_{readout} are unknown. While contribution from t_{exp} is known, t_{phase} has a known interval related to the frame period. t_{led} is the reference event used to timestamp the start of the measurement

- **LED activation** (t_{LED}) - LED turns on
- **Phase offset** (t_{phase}) - the time between LED activation and the exposure rows capturing photons from the LED.
- **Exposure time** (t_{exp}) - the interval during which a row captures photons.
- **Sensor readout** (t_{readout}) - analog-to-digital conversion and readout for one row.
- **CSI transfer** (t_{csi}) - transmission of pixel data from the camera to the Raspberry Pi.
- **Direct Memory Access (DMA) and buffer completion** (t_{dma}) - the Raspberry Pi writing the frame into a memory buffer.

The mentioned flow can be seen in Figure 4.3

4.2.2 LiDAR

The LiDAR test setup was based on the sensor's trigger signal. The trigger input was connected to a Raspberry Pi GPIO output, allowing the Raspberry Pi to initiate each measurement cycle. The sensor communication interface was connected to the Raspberry Pi so that the returned measurement data could be received and timestamped.

At the software level, a timestamp was recorded when the trigger signal was sent, and a second timestamp was recorded when the first response packet from the LiDAR was received. This provided the total measured latency from trigger to received data, including sensor response time, communication delay, and software overhead.

A digital analyzer was also connected to the trigger and communication lines. This allowed the trigger event and the following sensor response to be observed directly at the signal level, making it possible to compare the software-level measurement with the lower-level communication timing.

The messuring points

- **Trigger signal** (t_0) - Trigger signal sent.
- **Measurement start** (t_{emit}) - The time when trigger signal reached LiDAR and it makes a measurement.
- **LiDAR starts transmitting data** (t_{data}) - Data is being transmitted over UART to Raspberry Pi.
- **Raspberry Pi recives data** (t_1) - Data reached memory of the device and can be used.

The experimental setup for the LiDAR measurements is illustrated in Figures 4.4 and 4.5.

4.2.3 IMU

The IMU test setup was based on the sensor's data-ready interrupt signal. The sensor was connected to the Raspberry Pi through I^2C, while the Interrupt (INT) pin was connected to a GPIO input. When a new measurement was available, the interrupt signal was used to trigger the readout procedure.

On the Raspberry Pi, a timestamp was recorded when the interrupt was registered. The software then read the corresponding measurement over I^2C, and a second timestamp was recorded after the read operation had completed.

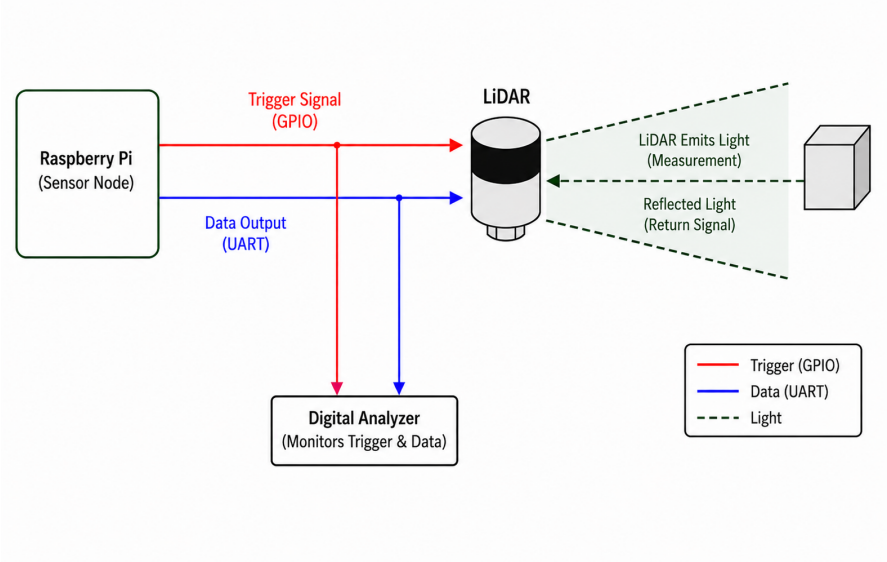


Figure 4.4: The practical set up for the LiDAR is shown here. This setup is using a trigger signal from the Raspberry Pi and measure time between signal sent until data comes back.

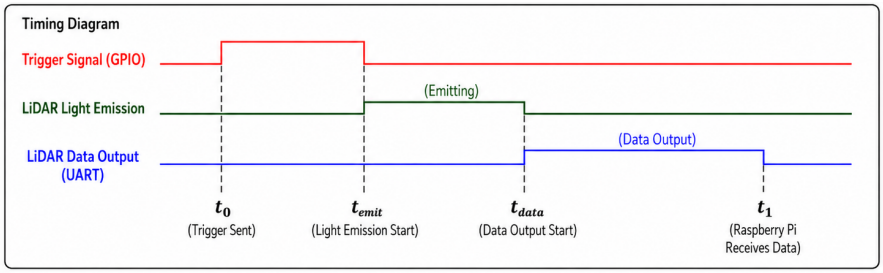


Figure 4.5: This figure shows the sequence of events that happens when the test runs. The trigger signal is sent and when the LiDAR receives the signal it starts the measurement, then it sends data back.

This provided the software-level latency from interrupt detection to completed sensor readout, including both communication time and code overhead.

A digital analyzer was also connected to the interrupt and I^2C lines. This made it possible to observe the interrupt signal and the following bus transaction directly, allowing the communication timing to be compared with the software-level measurement.

The setup and timing behavior of the IMU measurements are shown in Figure 4.6.

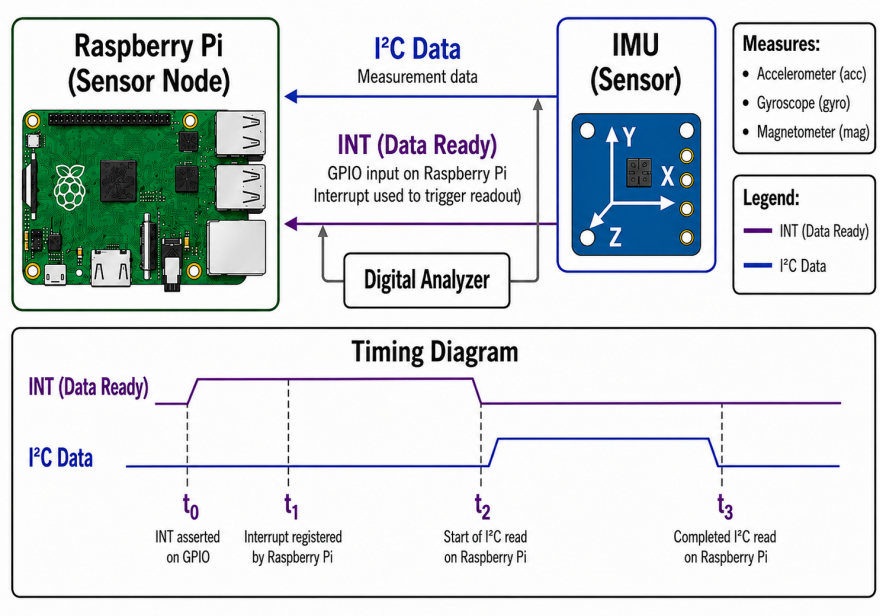


Figure 4.6: The figure shows the sequence of events that happens in the test. This figure is a high level overview not an exact representation. t_0 represents when the INT goes up, t_1 shows approximated when Raspberry Pi register the event. t_2 is when the Raspberry Pi start to read the data and t_3 is when it is finished reading.

4.2.4 IR Camera

The IR camera was more challenging to measure than the other sensors, since the image data is transmitted as a continuous video stream rather than as a single triggered sample. Therefore, the measurement points were placed inside the camera firmware and on the Raspberry Pi receiving side.

The firmware was modified to embed timing metadata into each UVC frame. Two timestamps were added on the camera board: t_{rx}, recorded when a full frame had been received from the Lepton sensor and was available in the board buffer, and t_{usb}, recorded just before the frame was packetized and sent over USB. On the Raspberry Pi, a timestamp t_{pi} was recorded when the corresponding frame was received in memory.

To compare timestamps from the camera board and the Raspberry Pi, a software clock synchronization procedure was implemented using repeated USB round-trip measurements.

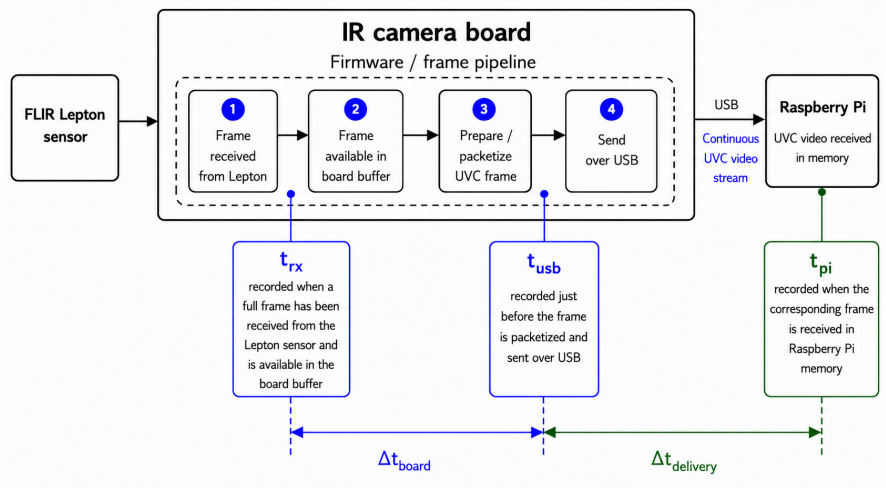


Figure 4.7: This figure provides an overview of the measurement pipeline used in the test. The measured sections are the internal processing delay on the camera board and the delivery time from the processed frame leaving the board until it reaches the Raspberry Pi.

The sample with the lowest round-trip time was used to estimate the clock offset between the two devices, and this offset was sent to the camera board so that board timestamps could be converted approximately into the Raspberry Pi time domain.

The board internal delay and delivery delay is defined as

$$\Delta t_{\text{board}} = t_{\text{usb}} - t_{\text{rx}} \tag{4.1a}$$

$$\Delta t_{\text{delivery}} \approx t_{\text{pi}} - t_{\text{usb}} \tag{4.1b}$$

Here, t_{usb}, t_{rx}, and t_{pi} denote the timestamps corresponding to frame transmission over USB, frame availability at the board from the camera, and frame availability at the Raspberry Pi, respectively. The delivery delay includes the USB transfer, operating system handling-, buffering, and reception by the Raspberry Pi application. It should be interpreted as an estimate because the two clocks were aligned using software rather than a shared hardware clock. The setup and timing behavior of the IR measurements are shown in Figure 4.7.

4.2.5 Cloud server

Kubernetes was used as the receiver for the device. The implementation involved creating a container in which a Python server was running. A Kubernetes service also had to be configured in order to expose the container and allow the data to be routed correctly. The server was designed to receive data from the device and then send the data back.

A key limitation of this implementation was the limited number of available measurement points within the cluster. The arrival time of a frame at the container could be measured. However, the clocks on the Raspberry Pi and the server were not synchronized. Therefore, Round-Trip Time (RTT) was used as the primary measurement metric.

Furthermore, the network path within the cluster is not direct. The data first passes through a proxy, which forwards it to a Kubernetes service. The service then relays the data to the appropriate connected container. These factors made the estimation more challenging and made the network path difficult to map accurately.

4.2.6 Network

The network component of the testbed was implemented using a cloud server and a 5G HAT mounted on the Raspberry Pi 4. This configuration enabled data transmission from the sensor node to the cloud server over the cellular network.

The network experiment focused on measuring the RTT for video frame transmission. A timestamp was recorded when a video frame was sent from the sensor node and the RTT was calculated when an acknowledgment response was received from the cloud server. This measurement was used to evaluate the communication delay introduced by the cellular network and cloud connection.

Only video frames were used in this experiment. Data from the IMU and LiDAR were not included in the network measurements. This decision was made because video frames are significantly larger than the data produced by the other sensors. Therefore, the video stream was considered the most demanding part of the sensor data transmission and the most likely bottleneck in the network communication.

The RTT is defined as

$$\Delta t_{\mathrm{RTT}} = t_{\mathrm{ack}} - t_{\mathrm{send}}. \tag{4.2}$$

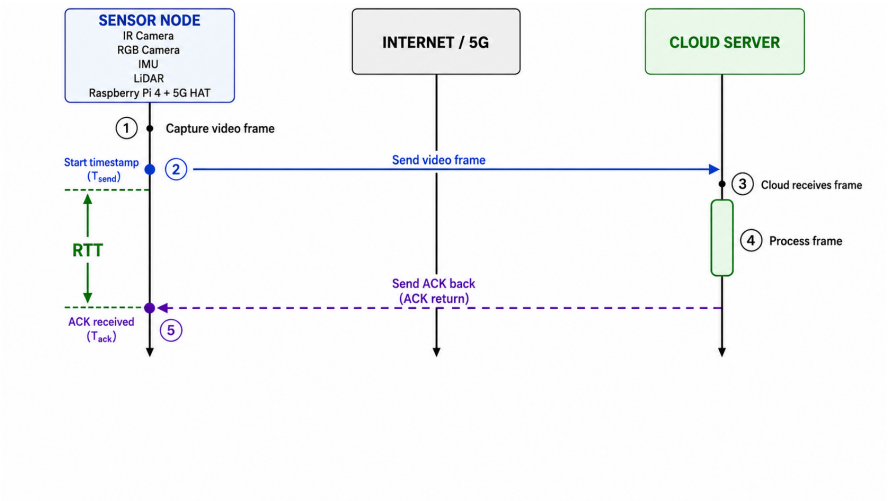


Figure 4.8: The figure shows the timeline of events that occur during the test. The frame is captured by the camera and right before the frame is being sent to the modem we start the clock. Sending it to the cloud service where it is then being processed and finally a ACK is sent back and that is were the clock stops. The RTT is the total time from that a frame is being sent until an ACK is sent back.

Here, t_{ack} denotes the timestamp at which the Acknowledgment (ACK) signal is received from the server, and t_{send} denotes the timestamp at which the frame is sent. The setup and timing behavior of the IR measurements are shown in Figure 4.7. The setup and time sections are shown in Figure 4.8.

Chapter 5

Results and Analysis

This chapter presents the results obtained for each sensor used in the thesis. The results are presented separately for each sensor to provide a clear overview of the measured latency values and the behaviour observed during the measurements.

In addition to presenting the results, the chapter includes an analysis of the reliability and validity of the measurements. The reliability analysis considers how dependable the results are based on the measurement procedure, timing references, and observed variation in the data. The validity analysis considers whether the measurements represent the intended timing points and whether the results are suitable for supporting the overall conclusions of the thesis.

5.1 Sensor and Transmission Results

This section presents the main results from the conducted experiments. The results are divided into subsections for each sensor and for the network transmission tests. This makes it possible to highlight the most important findings from each part of the sensor node and to provide an overview of the result for respective sensor.

5.1.1 LiDAR

The LiDAR latency was evaluated by measuring the time between the trigger command, the transmitted request message, and the received response from the sensor. This allowed the total software-to-response latency to be separated from the sensor response time and the surrounding code overhead.

Table 5.1: LiDAR latency statistics for 20 trigger signals. The listed time sections represent different parts of the trigger-to-response pipeline. For all measurement points, the standard deviation was 0.008 ms. The main contributor to the measured variation was the LiDAR response time, since it was included in all measured sections of the pipeline.

Time section	**Average (ms)**	**Median (ms)**	**Min (ms)**	**Max (ms)**
Total, including code section	11.053	11.051	11.045	11.066
Total, excluding code section	10.919	10.915	10.909	10.932
Trigger sent–message sent	10.654	10.653	10.645	10.667
LiDAR response time	9.959	9.958	9.949	9.972

Table 5.2: IMU latency statistics for 20 samples. The confidence interval is the 95% CI of the mean.

Time section	**Average (ms)**	**95% CI (ms)**	**Median (ms)**	**σ (ms)**	**Min (ms)**	**Max (ms)**
Total, including software	1.838	$\pm$0.026	1.839	0.056	1.854	1.869
I^2C transaction only	1.373	$\pm$0.003	1.337	0.006	1.328	1.434

The complete measurement includes the full code section, from the point where the trigger is initiated until the response has been processed. The reduced measurements separate the communication and sensor response stages to show where the main latency occurs. The results are summarized in Table 5.1.

5.1.2 IMU

The IMU latency was evaluated by observing the I^2C communication between the Raspberry Pi and the sensor using a digital analyzer. This made it possible to measure the time spent on the bus transaction separately from the time spent in the surrounding software section.

Two latency values are reported. The first includes the complete measured code section, from the point where the software initiates the sensor read until the result is available. The second excludes the surrounding code section and focuses on the I^2C communication itself. The results are summarized in Table 5.2.

Table 5.3: Summary of measured IR camera latency sections, for 200 frames.

Time section	**Average (ms)**	**Median (ms)**	σ **(ms)**
Camera board – Raspberry Pi	75.036	75.114	9.216
Board processing time	0.258	0.258	0.001

5.1.3 IR Camera

The IR camera latency was evaluated using two measurement points in the camera pipeline. First, the internal board processing delay was measured as the time between a completed frame being available in the camera board buffer and the point where the same frame was packetized for USB transmission. Second, the UVC frame delivery delay was estimated as the time from USB packetization on the camera board to the point where the corresponding frame was received by the Raspberry Pi application.

Table 5.3 summarizes the average, median, and standard deviation for these two sections. Figure 5.1 shows the estimated UVC frame delivery delay for each received frame during one 200-frame measurement run. The figure illustrates that the delivery delay is not constant for every frame, but remains within a relatively stable range over the measurement period.

The results show that most of the IR camera latency is introduced between the camera board and the Raspberry Pi, with an average delay of 75.036 ms and a standard deviation of 9.216 ms. In comparison, the internal board processing time is very small and stable, with an average of 0.258 ms and a standard deviation of only 0.001 ms. This indicates that the USB/UVC delivery stage dominates the measured IR camera latency, while the board-side processing contributes only marginally. The periodic pattern visible in Figure 5.1 is likely caused by the interaction between the fixed IR frame period and a periodic receive or dequeue timing component on the Raspberry Pi side. Since the Raspberry Pi timestamp is taken after the frame has passed through the USB/UVC stack and has been returned to the Raspberry Pi application, the measured delay includes not only transfer time but also any waiting time before the frame is delivered to the application. As the camera frame timing and the Raspberry Pi receive timing are not phase-locked, this waiting time changes from frame to frame and wraps periodically, producing the observed oscillating shape.

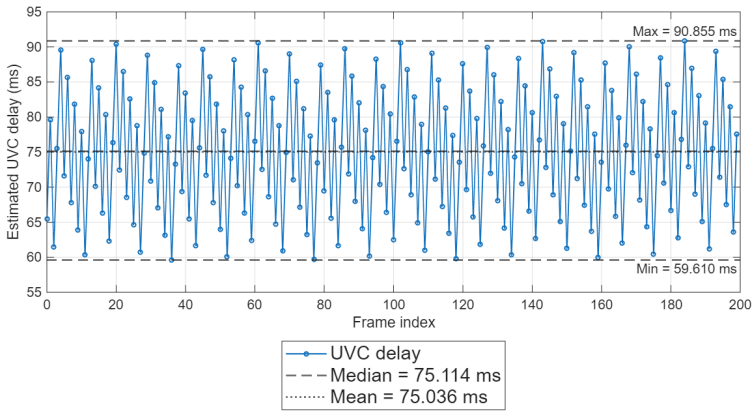


Figure 5.1: Estimated UVC frame delivery delay from camera board USB packetization to Raspberry Pi's memory reception over 200 frames.

5.1.4 Photon-to-frame-in-RAM pre ISP

The presented results visualize the sorted trails, the distribution, the mean decomposition between pipeline latency and phase offset, and the summarized latency numbers achieved with the frame resolution of 1536 × 864 and the measured frame rate of approximately 116 Hz.

Average latency decomposition as seen in figure 5.3 provides an estimate of how much of the total latency measured can be attributed to phase offset. The phase offset component is estimated to be half of the frame period, assuming a uniform distribution. It represents the expected waiting time until frame detecting event begins exposure.

5.1.5 Network Transmission via 5G

The average RTT for a complete pair of frames was measured to be 44.82 ms, while the median RTT was 28.5 ms. The standard deviation was 73.8 ms, which shows that the measured RTT values varied during the test. The results from the 5G network transmission test are summarized in Table 5.5.

The distribution of the measured pair RTT values is shown in Figure 5.4. The RTT variation over time is shown in Figure 5.6, where occasional latency spikes can be observed.

Packet loss during the 5G test was low. As shown in Table 5.5, the RGB stream had no lost packets, while the IR stream lost one packet.

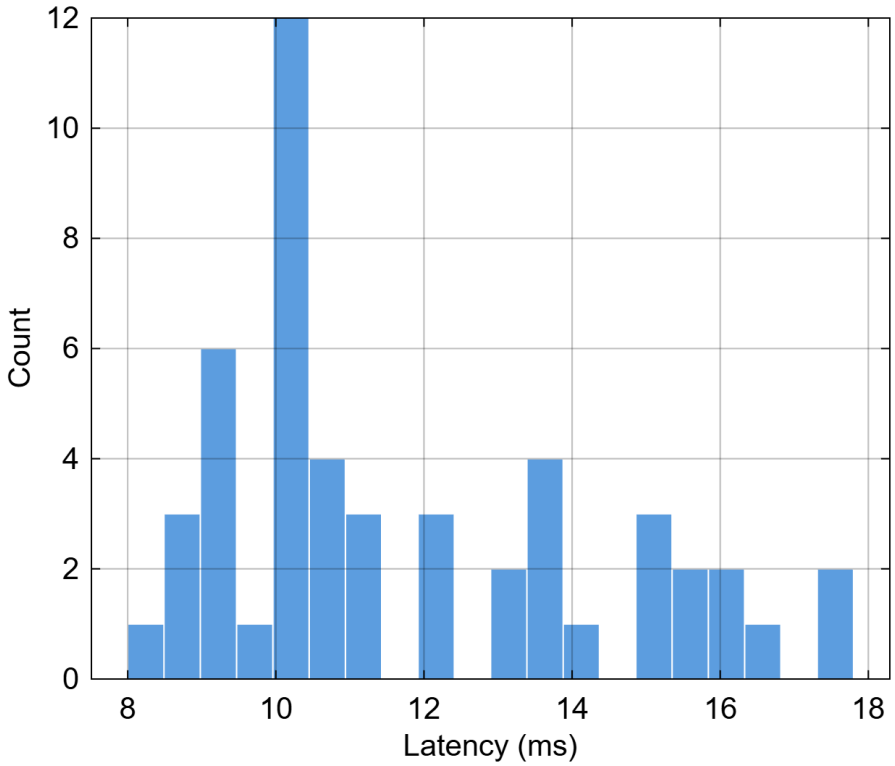


Figure 5.2: Latency distribution Photon-to-frame-in-RAM pre ISP latency from 50 trails.

Table 5.4: Summary of latency statistics pre-ISP for the Photon-to-frame-in-RAN experiement. The frame resolution was 1536×864, measured frame rate around 116 Hz.

Metric	**Value**
N (trials)	50
Minimum	8.384 ms
Median	10.666 ms
Mean	11.789 ms
95th percentile (p95)	16.317 ms
Maximum	17.745 ms
σ	2.562 ms
Frame period	8.600 ms
Average phase offset $E[t_{\text{phase}}]$	4.300 ms
Pipeline latency	7.489 ms

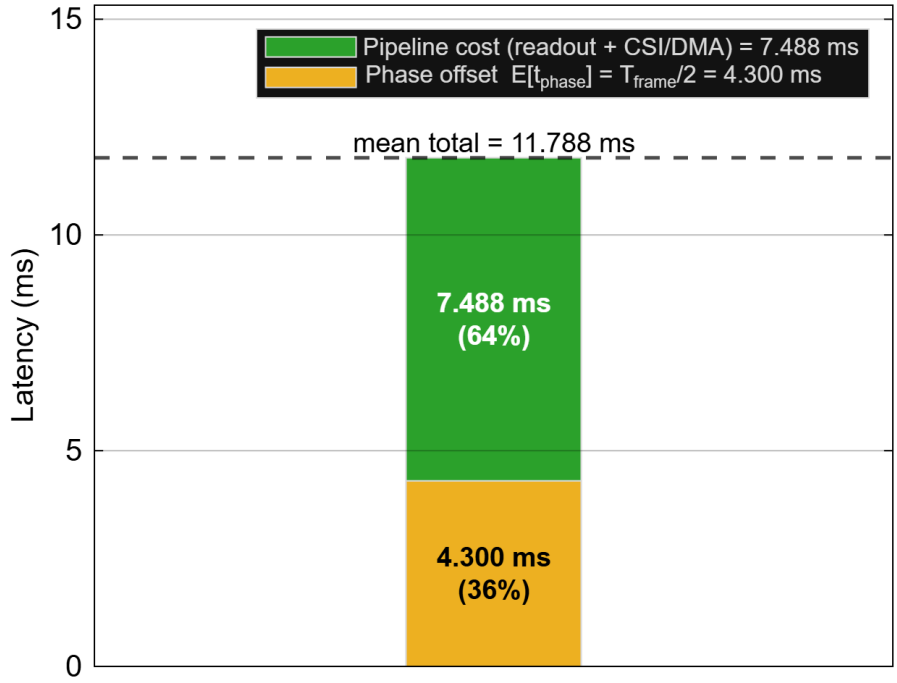


Figure 5.3: Mean latency decomposition Photon-to-frame-in-RAM pre ISP latency. Figure shows an estimation of latency contribution from phase offset and camera pipeline. Section 3.3.3 explains how the average phase offset is determined and how it impacts the overall latency.

Since the IR stream transmitted 400 packets during the measurement interval, this corresponds to an IR-specific packet loss of 0.25%. Across both camera streams, a total of 4000 packets were transmitted, giving an overall packet loss of 0.025%. The packet loss per frame is visualized in Figure 5.5. Overall, the results show that the implemented UDP-based transmission and acknowledgement method was able to deliver frame pairs over 5G with low packet loss and measurable round-trip latency.

For the additional no-loss dataset, the frame pair affected by packet loss was excluded and replaced with another valid frame pair from the recorded samples. This kept the dataset at 200 valid frame pairs, making it consistent with the original measurement size. The purpose of this dataset was to evaluate the latency of successfully completed frame pairs and to provide a comparison case without the influence of the packet loss event. The resulting values are summarized in Table 5.6.

Table 5.5: A summary of the network transmission test is presented here. In total, 200 frame pairs were transmitted, corresponding to 3600 RGB packets and 400 IR packets. During the test, one packet was lost.

Metric	Value	Percentage
Pair RTT average	44.8 ms	–
Pair RTT median	28.5 ms	–
Lost packets, RGB camera	0	0%
Lost packets, IR camera	1	0.25%
Overall packet loss	1	0.025%
Pair σ	73.8 ms	–

Table 5.6: The network transmission test consisted of 200 transmitted frame pairs, corresponding to 3600 RGB packets and 400 IR packets in total. To enable comparison and evaluate the effect of packet loss, the frame pair affected by packet loss was excluded from the analysis and replaced with another valid frame pair from the recorded samples. As a result, the statistical analysis was performed using 200 valid frame pairs.

Metric	Value	Percentage
Pair RTT average	39.95 ms	–
Pair RTT median	28.44 ms	–
Lost packets, RGB camera	0	0%
Lost packets, IR camera	0	0%
Overall packet loss	0	0%
Pair σ	23.32 ms	–

5.1.6 Network Transmission via Ethernet

The Ethernet transmission test was performed as a reference measurement using a wired connection between the Raspberry Pi and the receiving system. The measured results are presented in Table 5.7. For the complete frame pairs, the average RTT was 4.60 ms, the median RTT was 3.90 ms, and the standard deviation was 3.1 ms.

The measured RTT values over time are shown in Figure 5.8, while the distribution of the values is shown in Figure 5.7. The Ethernet connection was used as a reference case because it provides a wired communication path without radio scheduling or varying signal conditions.

No packet loss was observed during the Ethernet test. As shown in Table 5.7, both the RGB and IR streams transmitted all packets successfully during the measurement interval.

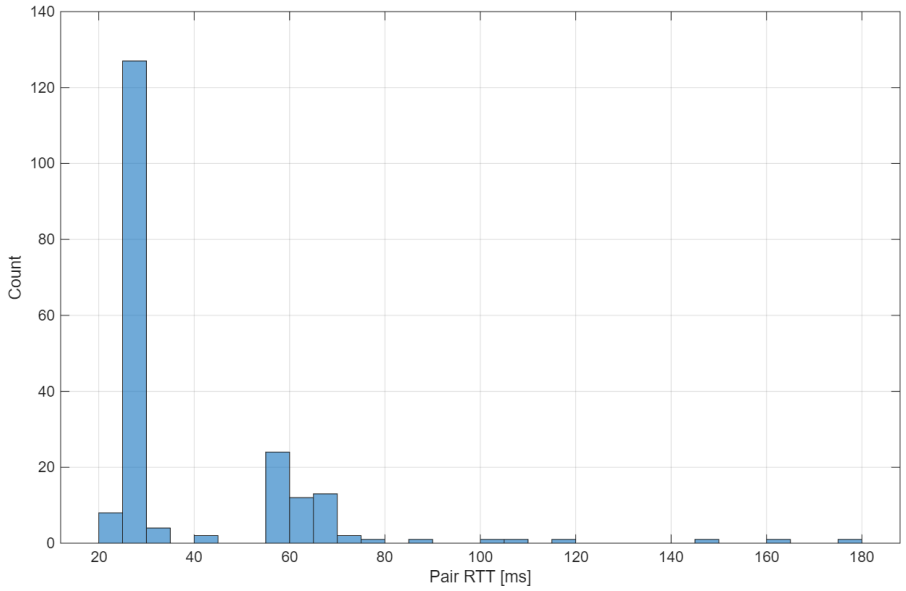


Figure 5.4: Histogram of RTT for 199 frame pairs transmitted over 5G, excluding one frame pair affected by packet loss. The data is grouped into 5 ms intervals, with the majority of samples falling between 25 ms and 30 ms.

This makes the Ethernet result useful as a no-loss reference measurement when comparing against the 5G transmission results.

Overall, the Ethernet results show that the implemented UDP-based transmission and acknowledgement method was able to deliver all frame pairs without packet loss over the wired connection. The result therefore provides a baseline for the transmission performance under controlled network conditions.

5.2 Reliability Analysis

Reliability was an important consideration in this thesis to ensure that the reported results were based on measurements that could be trusted. Since the measurements were collected through different sensors and measurement setups, it was necessary to consider how the chosen methods affected the dependability of the results.

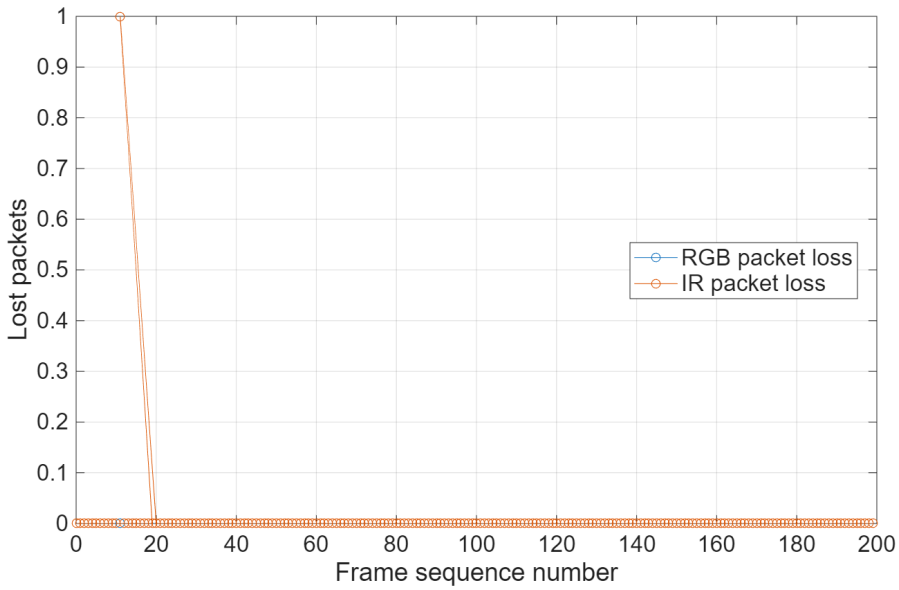


Figure 5.5: Packetloss for each frame on each camera, total loss of one frame. This amounts to a package loss of 0.025% over both cameras. Packet loss occurred at frame 11, transmitted over 5G.

Table 5.7: Summary of the network transmission test. In total, 200 frame pairs were sent, corresponding to 3600 RGB packets and 400 IR packets. Transmitted over Ethernet

Metric	**Value**	**Percentage**
Pair RTT average	4.60 ms	–
Pair RTT median	3.90 ms	–
Lost packets, RGB camera	0	0%
Lost packets, IR camera	0	0%
Overall packet loss	0	0%
Pair σ	3.1 ms	–

This chapter discusses how reliability was supported for each sensor by examining the measurement procedures, timing references, and sources of measurement uncertainty. The focus is on how the measurements were structured, how potential sources of error were reduced, and how additional verification methods were used where possible.

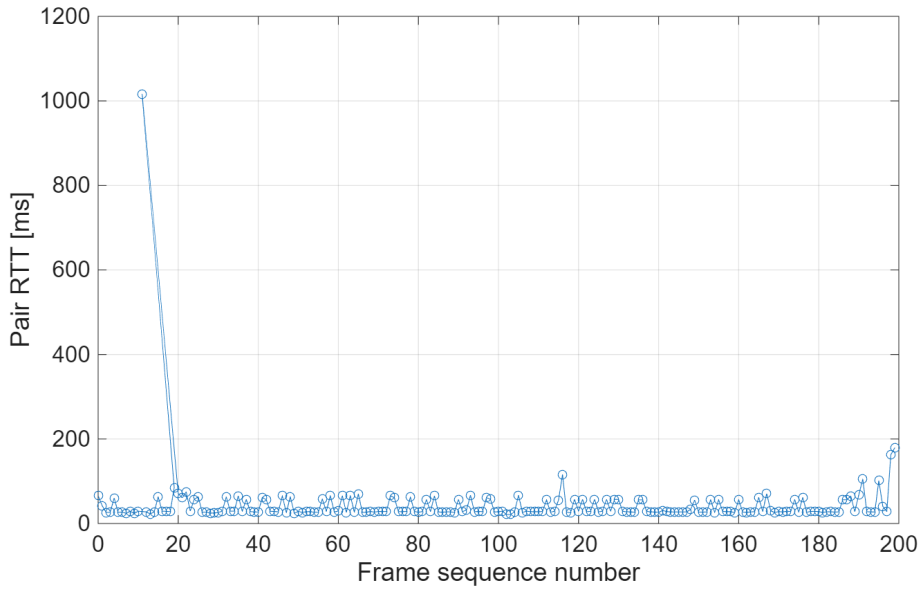


Figure 5.6: This figure shows the RTT for each frame pair, transmitted over 5G. It shows the occurrences of one packet being lost this was at frame 11.

By considering these aspects, the chapter provides a basis for evaluating the reliability of the latency results presented in the thesis.

5.2.1 LiDAR

The LiDAR measurements were performed using a structured test procedure based on the trigger signal supported by the sensor. The trigger signal was used as a defined start point for the measurement. From this point, the total latency was measured until the LiDAR measurement had been received and processed by the Raspberry Pi. Since this measurement was performed using only the Raspberry Pi clock, it avoids clock synchronization errors between separate devices.

To further improve the reliability of the result, a second measurement was performed using a digital analyzer. This allowed the transmitted signal to be observed directly on the communication line. By measuring the timing at the signal level, the software overhead from the Raspberry Pi code could be separated from the communication and sensor response time. This made it possible to compare the complete software-level latency with the lower-level

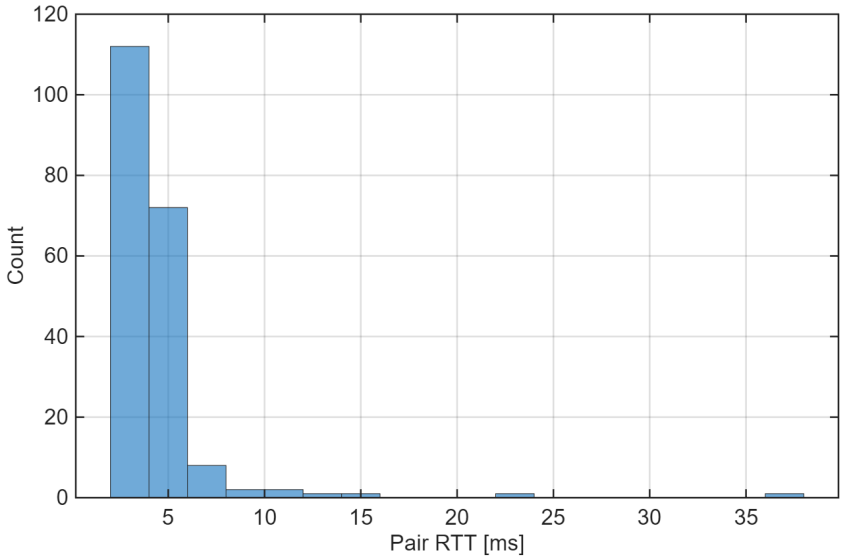


Figure 5.7: Histogram over RTT frames transmitted over Ethernet, sample size was 200 frame pairs.

bus timing.

5.2.2 IMU

The IMU measurements were constructed around the interrupt signal from the sensor. The interrupt pin was used as the start point for the software-level latency measurement. When the Raspberry Pi registered the interrupt, a timestamp was recorded, and the latency was measured until the corresponding sensor data had been read. This measurement therefore includes both the sensor readout time and the software overhead in the Raspberry Pi code.

A second measurement was performed using a digital analyzer. In this case, the interrupt pin and the I^2C bus were observed directly. This made it possible to measure the time from the interrupt signal becoming active until the data transfer on the I^2C bus started. By observing the signal level directly, this measurement reduces the influence of software overhead and gives a clearer view of the communication timing between the Raspberry Pi and the IMU.

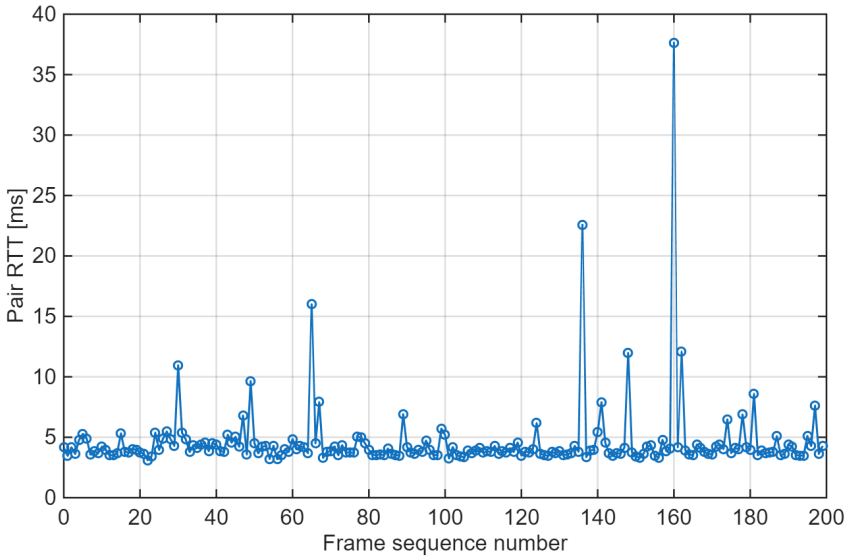


Figure 5.8: RTT for pair of frames, sample size was 200 frame pairs transmitted over Ethernet

5.2.3 IR Camera

The reliability of the IR camera measurement was strengthened by separating the latency into clearly defined stages instead of treating the camera stream as one single delay.

Since the camera transmits frames continuously using UVC, the measurement could not rely on a simple request-response timing method. Instead, timestamps were placed at relevant points in the frame pipeline, making it possible to distinguish between the delay inside the camera board and the delay introduced by USB transfer and the Raspberry Pi video pipeline.

A key part of the measurement reliability was the use of frame-embedded metadata. By embedding the timing information directly into each video frame, the timestamps remained associated with the exact frame being evaluated. This reduces the risk of comparing a Raspberry Pi receive timestamp with the wrong camera frame, which is especially important for a continuous video stream.

The board-side timestamps were also compared with the Raspberry Pi receive time using a software clock synchronization procedure. The synchronization was based on repeated USB round-trip measurements, where the lowest measured round-trip time was used to estimate the clock offset between the camera board and the Raspberry Pi. Using the lowest round-trip time reduces the influence of temporary operating system scheduling delays and USB queueing effects, making the clock offset estimate more reliable.

The measurement was repeated over multiple frames, which allowed the stability of the delay to be evaluated. The frame counter embedded in the metadata was used to verify that frames were received sequentially, reducing the risk that dropped or reordered frames affected the result. Together, the embedded timestamps, clock offset estimation, and repeated frame measurements provide a more reliable estimate of the practical UVC frame delivery delay.

5.2.4 Photon-to-frame-in-RAM pre ISP Latency

The experiment method and testbed are configured to make trail runs consistent in terms of environment, However, the method does not take thermal throttling or external load on the processing unit into account when running tests. The method uses a physical reference event, a timestamp is taken when the LED is triggered via GPIO. This reference is a genuine event and not a simulated software event, which adds reliability around triggering the event. However, the GPIO timestamp is recorded in memory which introduces some variable overhead from running software and OS scheduling.

5.2.5 Network Transmission

The reliability of the network transmission results depends on both the measurement method and the collected data. The method measured the complete transmission path using real camera data instead of artificial test packets. Each RGB and IR frame was encoded, divided into UDP packets, transmitted to the receiver, and acknowledged after reception. Therefore, the measured RTT values represent the end-to-end behaviour of the implemented communication method rather than only the theoretical latency of the network link.

Each transmitted frame was assigned a unique frame identifier, which was included in every packet and returned in the acknowledgement. This allowed each acknowledgement to be matched to the correct transmitted frame and reduced the risk of incorrect RTT calculations caused by delayed, duplicated, or out-of-order packets. Since the RGB and IR streams were transmitted on separate UDP ports, the shared identifier also made it possible to match the two streams into complete frame pairs.

The receiver counted how many packets were received for each frame before sending an acknowledgement. This made it possible to detect incomplete frames and calculate packet loss from the actual number of transmitted and received packets. The same measurement procedure was used for both the 5G and Ethernet tests, which made the comparison more reliable.

The data was evaluated using average and median RTT values, together with packet loss per stream and overall. These metrics give a more robust interpretation than relying on a single latency value. The results should therefore be interpreted as frame-level end-to-end RTT measurements, not pure network latency, since they also include processing time in the Raspberry Pi and receiver.

5.3 Validity Analysis

Validity was carefully considered for all sensors used in this study to ensure that the collected data accurately represented the intended measurement points within the latency pipeline. Since this thesis focuses on decomposing sensor latency across different stages of the pipeline, the correctness of each sensor output is essential for identifying where latency is introduced and how it propagates through the system.

This chapter discusses how validity was supported for each sensor. The focus is on whether the measurements reflect the intended timing behaviour,

how potential sources of error were considered, and how the collected data supports the overall latency decomposition.

5.3.1 LiDAR

The validity of the LiDAR latency measurements is supported by using the sensor trigger signal as a clearly defined start point. Since the Raspberry Pi generated the trigger and recorded the software timestamps, the latency could be measured from the moment a measurement was requested until the first response packet was received.

At the software level, timestamps were recorded when the trigger signal was sent and when the first response packet arrived. This captures the practical latency of the implemented system, including sensor response time, communication delay, and Raspberry Pi software overhead. A digital analyzer was also used to observe the trigger and communication lines directly, providing a lower-level reference that is less affected by operating system scheduling and software timing delays.

Together, the software timestamps and digital analyzer measurements strengthen the validity of the results by showing both complete system latency and physical signal-level timing. However, the results should be interpreted as system-specific latency values, since they depend on the Raspberry Pi, GPIO handling, communication interface, sensor configuration, operating system, and implemented readout code.

5.3.2 IMU

The validity of the IMU latency measurements is supported by using the sensor's data-ready interrupt as a clearly defined starting point. Since this interrupt indicates that new measurement data is available, it provides a relevant reference for measuring the delay until the corresponding data readout is completed.

At the software level, the Raspberry Pi recorded one timestamp when the interrupt was detected on the GPIO input and another timestamp after the I^2C read operation had finished. This captures the practical latency of the implemented system, including interrupt handling, software execution, I^2C communication, and sensor readout time.

A digital analyzer was also used to observe the interrupt signal and the I^2C bus directly. This provided a lower-level timing reference that is less affected by operating system scheduling, timestamp resolution, and software overhead.

Together, the software timestamps and digital analyzer measurements strengthen the validity of the results by showing both complete system latency and physical bus-level timing. However, the results should be interpreted as system-specific latency values, since they depend on the Raspberry Pi, GPIO handling, I^2C configuration, operating system, sensor configuration, and implemented readout code.al setup, but it may change if the bus speed, software implementation, sensor configuration, or system load is modified.

5.3.3 IR Camera

The validity of the IR camera latency measurements is supported by placing the measurement points within the actual frame delivery pipeline. By dividing the latency into separate stages, the delay introduced by the camera board could be separated from the delay caused by USB transfer and the Raspberry Pi video pipeline.

A key factor was the use of frame-embedded metadata. Since the IR camera transmits frames continuously using UVC, timing information and a frame counter were embedded directly into each frame. This ensured that timestamps from different parts of the pipeline were associated with the same frame, reducing the risk of comparing unrelated events.

Clock synchronization between the camera board and the Raspberry Pi was handled by estimating the clock offset using repeated USB round-trip measurements. The lowest measured round-trip time was used, as it is least affected by temporary delays such as operating system scheduling and USB queueing.

The measurements were repeated over multiple frames, and the embedded frame counter was used to verify sequential frame reception. This strengthened the validity of the results by showing that the measured latency was consistent over time and not caused by dropped or reordered frames.

However, the results should be interpreted as system-specific latency values. The measured delay depends on the camera board, UVC transfer, USB connection, Raspberry Pi video pipeline, camera configuration, frame rate, resolution, system load, and implemented processing pipeline.

5.3.4 Photon-to-frame-in-RAM pre ISP Latency

The test can be deemed valid for measuring latency between camera acquisition and availability of RAW sensor data in RAM, since the optical event generated by the GPIO-controlled LED is directly observed by the

camera system. However, the measurement includes additional delays introduced by exposure timing, rolling-shutter readout, buffering, and therefore does not isolate sensor latency alone.

The results are only valid for the frame resolution and frame rate used in the benchmark runs.

5.4 Network Transmission

The network transmission measurements were performed using real image data from the two cameras instead of only artificial test packets. This makes the measurements more representative of the intended system, since the transmitted data consisted of encoded camera frames divided into UDP packets. The same transmission and acknowledgement method was used for both the 5G and Ethernet tests, which makes the results comparable between the two network configurations.

Each RTT value was linked to a specific frame or frame pair using a frame identifier. This allowed each acknowledgement to be matched with the correct transmitted data and reduced the risk of mixing measurements from different frames. Since the RGB and IR streams were transmitted on separate UDP ports, the frame identifier was also used to match the corresponding RGB and IR frames into a complete frame pair.

The 5G test represents the intended wireless transmission path to the cloud environment, while the Ethernet test was used as a wired reference measurement. Including both network types makes it possible to compare the behavior of the transmission method under different communication conditions. The Ethernet measurement provides a baseline where effects from the wireless 5G link, such as radio scheduling and signal variation, are not present. Differences between the 5G and Ethernet results can therefore give an indication of how much of the measured latency variation and packet loss is related to the wireless network path compared with the local transmission implementation.

The method also makes it possible to evaluate both stream-specific and pair-based performance. The individual RGB and IR acknowledgements show the latency and packet loss for each camera stream, while the pair RTT represents the time required for both streams in a frame pair to be acknowledged. This is relevant because the system depends on receiving data from both cameras, not only one individual stream.

Chapter 6

Discussion

This chapter provides an in-depth discussion of the results obtained for each sensor used in the study. The discussion focuses on interpreting the sensor-specific results, identifying patterns in the measured latency, and evaluating what these findings indicate about latency in the system.

Furthermore, the chapter addresses the main limitations of the results and considers how these limitations may affect their interpretation. Possible improvements are also discussed, including refinements to the measurement setup, and system design that could support more accurate latency analysis in future work.

6.1 Sample Size

The sample sizes for the different sensors were previously discussed in Section 3.3.4. The smaller sample sizes used for the IMU and LiDAR introduce some uncertainty. However, the reported metrics in Tables 5.2 and 5.1 show that the measured values are close together. This is also supported by the confidence intervals, as the margins of error are small for both sensors. For future work, the focus for these two sensors should not primarily be on increasing the sample size, but rather on expanding the current testbed to provide more points for collecting latency data.

The RGB camera was given more time and focus compared to the IMU and LiDAR, as the camera data was of greatest interest to Ericsson. The choice of sample size was mainly related to the time required to conduct the tests. Running the RGB latency benchmark 50 times still required a considerable amount of time, and therefore it was decided to limit the test to 50 runs. While this sample size is sufficient for estimating mean and median latency, it does

limit the confidence of the tail latencies such as maximum. A larger sample size could potentially provide more accurate results, and increase observing latency spikes potentially caused by jitter.

The IR camera had the most uncertainties, mainly due to the synchronization between the internal clocks of the Raspberry Pi and the thermal board. Therefore, a larger sample size was chosen in an attempt to obtain more reliable readings. A larger sample size might have yielded better results. However, we believe that in future works that the main focus should instead be on improving the synchronization between the two clocks.

The network tests used a large sample size in order to make the results as reliable as possible, since this was one of Ericsson's main points of interest. Since RTT was measured instead of one way latency, the results were expected to vary more, as cellular networks are known to vary in reliability. Therefore, a large number of samples was used for the RTT measurements.

6.2 LiDAR

The LiDAR latency results show that the measurements were stable across the tested sections. As shown in Table 5.1, the total latency including the code section had an average of 11.053 ms and a median of 11.051 ms. The close agreement between these values suggests that the measurements were not strongly skewed. In addition, the limited range between the minimum and maximum values as well as the standard deviation indicates that the measurements were concentrated around the central value, supporting the interpretation that the LiDAR latency results were reliable under the tested conditions.

The results should be interpreted as the latency of the complete test setup rather than the pure internal latency of the LiDAR. This is because the measurement includes the trigger command, communication timing, sensor response, and software handling. The total latency excluding the code section was 10.919 ms, while the measured LiDAR response time was 9.959 ms. This shows that most of the latency came from the sensor response and communication stages, while the surrounding code overhead was relatively small.

Overall, the LiDAR pipeline showed stable latency performance in the tested setup. However, the measured values are setup-specific and should not be interpreted as sensor-only latency values. Future testing could use a physical testbed with a moving object and an external trigger, as illustrated in Figure 6.1, to measure the latency from a real scene change to the availability

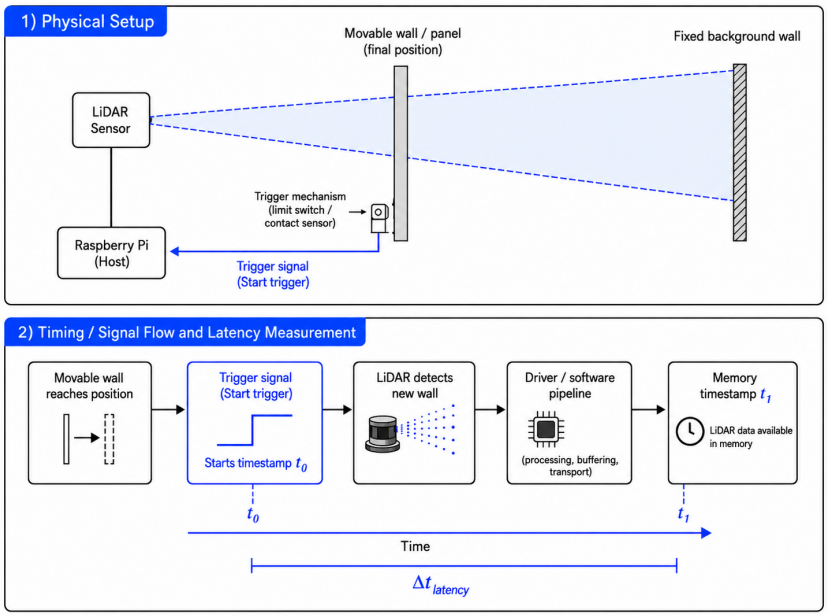


Figure 6.1: This figure illustrates a theoretical testbed setup for measuring the real-time event-to-memory latency of a LiDAR sensor. The setup uses a moving object that is positioned in front of the LiDAR. An additional reference sensor is used to trigger the timing measurement when the object reaches the defined position in front of the LiDAR.

of new LiDAR data.

For such a setup, the latency can be defined as

$$\Delta t_{\text{latency}} = t_1 - t_0. \tag{6.1}$$

Here, t_0 denotes the timestamp at which the trigger signal is received from the switch, while t_1 denotes the timestamp at which the corresponding sensor data is available in memory. This definition can also be applied to other event-to-memory measurements, such as the IR camera pipeline, where the setup and timing behavior are shown in Figure 4.7.

6.3 IMU

The IMU results show that the sensor readout latency was low and stable. As shown in Table 5.2, the total latency, including the software section, had an average of 1.838 ms and a median of 1.839 ms. The close agreement

between these values suggests that the measurements were not strongly skewed. Furthermore, the low standard deviation and the limited range between the minimum and maximum values indicate that the measurements were concentrated around the central value. This supports the reliability of the measured IMU latency under the tested conditions.

The I^2C transaction accounted for most of the measured delay, with an average latency of 1.373 ms. The remaining delay was caused by software overhead, such as interrupt handling, timestamping, and code execution around the read operation.

Factors that may have affected the result include the I^2C bus speed, the amount of data read from the sensor, and the Raspberry Pi software environment. Operating system scheduling and GPIO interrupt handling can also introduce small variations in the total latency.

Overall, the IMU latency was stable and relatively low in the tested setup. However, the measured values are setup-specific and should not be interpreted as the pure internal latency of the IMU. Future testing could use a physical testbed with a controlled motion event and an external trigger, as illustrated in Figure 6.2, to measure the latency from a real physical event to the availability of the corresponding IMU data in the Raspberry Pi's memory.

For such a setup, the latency can be defined as

$$\Delta t_{\text{latency}} = t_2 - t_0. \tag{6.2}$$

Here, t_0 denotes the timestamp at which the external trigger signal is received, while t_2 denotes the timestamp at which the corresponding IMU data is available in memory.

6.4 IR Camera

The IR camera results show that the total frame delivery delay is affected by several parts of the camera pipeline. The delay is not only caused by the camera board itself, but also by the UVC transfer, USB communication, buffering, and the Raspberry Pi video pipeline. This is important when interpreting the results, since the measured latency represents the practical delay of receiving frames in the Raspberry Pi application rather than only the internal delay of the camera hardware.

The results also show that the UVC frame delivery delay varied between frames. This variation can be explained by factors such as USB scheduling, buffering in the video pipeline, operating system scheduling on the Raspberry

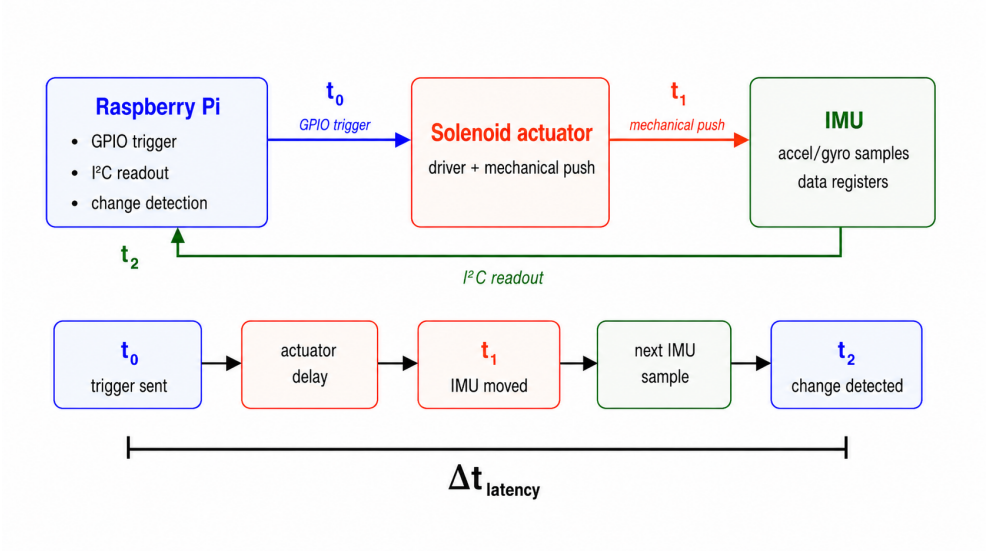


Figure 6.2: This figure illustrates a theoretical testbed for measuring the real-time event-to-memory latency of an IMU. The setup uses a triggerable push device that applies a mechanical impulse to the IMU. When activated, the device sends a trigger signal and causes the IMU to move. The Raspberry Pi monitors the incoming IMU data until motion is detected. The latency is then calculated as the time difference between the trigger signal and the detected motion.

Pi, and how frames are handled by the UVC driver.

Since the camera transmits frames continuously, frames may also be affected by queueing before they are made available to the application. Therefore, the measured delay should be interpreted as the latency of the full camera-to-application path.

The use of frame-embedded metadata made it possible to match each received frame with the corresponding board-side timestamp. This was important because the IR camera does not operate as a request-response device, where a single request can easily be matched to a single response. Without embedded metadata, it would be difficult to know if the timestamp comparison was made for the correct frame. This improves the usefulness of the results when discussing the real frame delivery behaviour.

A possible source of uncertainty is the clock synchronization between the camera board and the Raspberry Pi. The offset was estimated using repeated USB round-trip measurements, and the lowest measured round-trip time was used to reduce the influence of temporary delays. However, this method cannot completely remove all synchronization error. Some uncertainty may remain due to USB queueing, operating system scheduling, and small variations in when timestamps are recorded.

Overall, the IR camera measurement shows that the camera stream could be received with a stable but not constant delivery delay. The results indicate that the main delay is related to the complete UVC delivery path rather than only the camera board processing time. Further improvements could include reducing buffering, testing different frame rates and resolutions, and investigating how the Raspberry Pi video pipeline affects when frames become available to the application.

6.5 Photon-to-frame-in-RAM pre ISP Latency

The latency results received are high relative to the application field of XR. As mentioned in the implementation chapter 4.2.1, the measured latency consists of sequential sub-modules: phase offset, exposure time, sensor readout, CSI transfer and DMA. Sub-modules like exposure time could be manipulated to increase or decrease the time a row captures photons. Given that the exposure time does not exceed frame interval or is too short for any row to capture an impactful amounts of photons.

Photons from the LED may be captured in an early row, however the frame is still not available. Instead additional latency from remaining row exposure and readout is added until the frame can be accessed in RAM. In addition to this, the script determining if the LED event was captured or not inspects a full

frame. Therefore the results should rather be interpreted as end-to-end frame acquisition latency rather than the individual pixel row response time.

An important consideration to highlight is that the phase offset component is not measured, but instead estimated from the frame period. The estimation assumes LED activation occurs uniformly with respect to the camera frame cycle.

The latency measurements rely on comparing timestamps from the LED activation and the start-of-frame which originate from different kernel clock domains. Both clocks originate from the same Raspberry Pi hardware clock source; however, to compare them an offset calculation is required. Inaccuracy, drift, or jitter could introduce errors in the reported latencies, something that should be considered when interpreting the results.

The benchmark requires the LED to be centered towards the camera-sensor, visual inspections where carried out to make sure the LED was still centered between runs. But no exact measurements where carried out, which could have impacted the results if the LED had shifted slightly from the camera center. The results are only valid or to be compared against a camera pipeline with a similar frame rate, given that the frame rate directly impacts the frame duration, which impacts latency.

6.6 Network Transmission

The network transmission tests showed that the Raspberry Pi was able to transmit frame data from both cameras and receive acknowledgements over both Ethernet and 5G. The 5G test represents the intended wireless cloud transmission path, while the Ethernet test provides a wired reference.

The 5G results fulfilled the packet loss benchmark values in Table 2.1. During the test, 200 frame pairs were transmitted, corresponding to 3600 RGB packets and 400 IR packets. One IR packet was lost, giving an IR-specific packet loss of 0.25% and an overall packet loss of 0.025%. Since no complete frame pair was lost, the reliability goal was still satisfied.

When the packet loss event was included, the average pair RTT was 44.82 ms, the median was 28.5 ms, and the standard deviation was 73.8 ms. The high standard deviation was mainly caused by the lost packet, since the receiver timeout was set to 1 s. When a packet was missing, the receiver waited until the timeout expired before finalizing the frame pair, which created a large latency spike. This affected the average and standard deviation more than the median, since most frame pairs still had low latency.

An additional 5G result was calculated by excluding the frame pair affected by packet loss and replacing it with another valid frame pair from the recorded samples. This kept the analysis at 200 valid frame pairs. Without the packet loss event, the average pair RTT decreased to 39.95 ms and the standard deviation decreased to 23.32 ms, showing the effect that a single timeout event can have on the latency statistics.

The measured 5G result ended up between the two benchmark values in Table 2.1 in terms of speed. However, this comparison should not be interpreted as a direct one-to-one comparison, since the benchmark measurements were performed using different setups and measurement conditions. In addition, this thesis measured RTT, while the benchmark values were based on one-way latency. Therefore, the comparison should mainly be seen as an indication of relative performance rather than an exact equivalence.

The Ethernet test had no packet loss and is therefore most directly comparable to the 5G no-loss result. Its average pair RTT was 4.60 ms, the median was 3.90 ms, and the standard deviation was 3.1 ms. The lower average and standard deviation show that the wired connection provided a more stable transmission path, while the 5G path introduced additional latency variation due to radio conditions, scheduling, buffering, cloud processing, and network load.

Overall, the implemented UDP-based transmission and acknowledgement method worked over both Ethernet and 5G. The results show that the system can meet the benchmark goals under wireless cloud transmission, although timeout handling and varying 5G network conditions can noticeably affect the measured latency variation. It should be taken into account that the network transmission module is made up of several sub-modules, the 5G-modem on the sensor node, the 5G infrastructure and the cloud server. To get a clearer insight and more accurate results, the network transmission module should be modularized further.

Chapter 7

Conclusions and Future Work

This chapter presents the overarching conclusions of the thesis and summarises the main findings from the conducted work. The conclusions are based on the results obtained from the sensor latency analysis and highlight how the thesis contributes to a better understanding of latency within the studied system.

The chapter also discusses the limitations of the thesis and presents possible directions for future work. These limitations and future improvements provide context for how the results should be interpreted and how the work could be further developed.

Finally, the chapter concludes with a reflection on the broader impact of the thesis. This includes how the results may benefit NEXSoS and how the work can contribute to progress towards *SDG 11: Sustainable Cities and Communities*.

7.1 Conclusion

The purpose of this thesis was to investigate the latency behavior of a Raspberry Pi-based multi-sensor edge node intended for real-time situational awareness applications, as part of the broader NEXSoS project. The work focused on creating a modular view of the latency pipeline, including sensor acquisition, local processing, encoding, packetization, and network transmission.

A prototype was implemented using multiple sensing modalities, including RGB camera data, IR camera data, LiDAR, and IMU. The system was also extended with a network transmission pipeline for sending camera frame data to a remote receiver over Ethernet and 5G. The prototype was able to capture and handle data from the included sensors, and the camera streams

were successfully transmitted using the implemented UDP-based transmission and acknowledgement method. Therefore, the goal of developing a working multi-sensor prototype was partly achieved.

The goal of decomposing and quantifying latency between the sensor node and a remote system was partly achieved. The RGB camera pipeline was evaluated using a photon-to-frame-in-RAM method, while the IR camera pipeline was evaluated by measuring the delay from the camera board-side timestamp to reception in Raspberry Pi memory. These measurements showed that camera latency is affected not only by the sensor hardware, but also by transfer protocols, buffering, driver behaviour, and software processing. For the LiDAR and IMU, initial latency related test setups were developed, but complete physical-event-to-memory latency measurements were not fully established.

The network transmission measurements evaluated the implemented UDP-based method, which used frame identifiers and acknowledgements to calculate RTT for complete frame pairs and detect missing packets. The Ethernet test was used as a reference for the 5G test. During the 5G test, 200 frame pairs were transmitted, corresponding to 3600 RGB packets and 400 IR packets. One IR packet was lost, giving an overall packet loss of 0.025%.

When the packet loss event was included, the average pair RTT was 44.82 ms and the median pair RTT was 28.5 ms. Although these values were within the selected benchmark range, the comparison should not be interpreted as a direct one-to-one comparison, since the benchmark values were based on different setups and measurement conditions. In addition, this thesis measured RTT, while the benchmark values were based on one-way latency.

Therefore, the network latency results did not fully verify the benchmark goals. Instead, they indicate that the implemented 5G transmission setup achieved latency in a relevant range, but that a dedicated one-way latency testbed should be implemented in future work to allow a fairer comparison with the benchmark values.

In conclusion, the thesis demonstrates that a Raspberry Pi based multi sensor node can be implemented and used to evaluate latency in several parts of a real-time sensor pipeline. While a complete latency model for all sensor modalities was not achieved, the work provides a practical foundation for modular latency analysis. The results show that the camera and network transmission stages can be measured with useful precision. Further work should focus on completing the remaining sensor specific latency measurements as well as expanding upon the network transmission testbed. The thesis therefore, contributes both an implemented prototype and practical

insight into the challenges of measuring end-to-end latency in heterogeneous sensor systems.

7.2 Limitations

A general limitation across all parts of the study was the code overhead introduced by the implementation. In addition, several components functioned as black boxes, which limited the possibility of modifying or analyzing the underlying software in detail. This restricted the extent to which the internal behavior of certain parts of the system could be evaluated.

Another limitation was that some parts of the pipelines could not be measured accurately due to insufficient measurement tools. As a result, certain sources of latency could not be isolated or quantified.

The test beds were generally simple to construct and did not require extensive hardware or complex setup procedures. However, the RGB camera setup was more dependent on stable physical positioning. In this case, both the RGB camera and the LED had to remain fixed and centered throughout the measurements. Any movement or misalignment could affect the accuracy and reliability of the latency results.

Overall, these limitations mean that the measured latency values should be interpreted as results from the complete implemented sensor node pipelines rather than as exact measurements of each individual hardware or software component.

7.3 Future Work

Future work could expand this project by including additional latency measurements and more complete end-to-end tests. One possible continuation would be to measure the latency from a real-time physical event until the event is available in memory on the Raspberry Pi. This would be relevant for the LiDAR, IMU, and IR camera pipelines, as it would provide deeper insight into the latency introduced by the sensor hardware, drivers, operating system, and software interfaces. Figures 6.1 and 6.2 show example testbeds for the LiDAR and IMU, respectively. To further test the RGB camera could include ISP latency for different frame resolutions and frame rates, since it is a sub-module part of the visual sequential latency pipeline, and should not be overlooked.

Another important continuation would be to measure the one-way latency of data transmission through a 5G cellular network. If possible, the latency

could also be mapped across different parts of the 5G network until the data reaches the cloud. This would make it possible to better understand where delays occur in the communication path between the sensor node and external systems.

Finally, future work could measure the end-to-end latency of the complete sensor node under full workload. Running all sensors and pipelines at the same time would provide a more realistic understanding of the total system latency and could help identify bottlenecks that are not visible when testing individual components separately.

7.4 Reflections

This thesis contributes to sustainability by supporting the development of sensor node systems that may improve firefighters' situational awareness during emergency operations. The thesis connects to *SDG 11: Sustainable Cities and Communities*, since better information about the surrounding environment can support safer and more resilient communities. This thesis was conducted under the umbrella of NEXSoS, and the project provided a foundation for further work on mapping latency in a sensor node. This latency mapping can be used as a guide to identify where optimization efforts should be focused, whether in the hardware or the network.

By mapping latency in the sensor node pipelines, the thesis identifies areas where future optimization may improve the reliability and usefulness of sensor data. Low and predictable latency is important because the information provided to firefighters should reflect the current situation as accurately as possible. The reusable test beds developed in this work can also support continued testing and development without requiring new measurement setups for each test. An ethical aspect of the project is the use of cameras, since image-based sensors may raise privacy concerns. In this thesis, the camera tests were performed in controlled environments and focused only on latency measurements. No personal information was required for the tests, and the collected data was limited to what was necessary for evaluating the sensor node pipelines.

References

[1] P. C. Jain, "Recent advances in next generation cellular mobile networks-5G, 5G-Adv., and 6G," in *2025 International Conference on Innovation in Computing and Engineering (ICE)*, 2025, pp. 1–6. DOI: 10.1109/ICE63309.2025.10984307.

[2] Vinnova, *Network-enabled exchange of critical information in systems-of-systems (NEXSoS)*, Accessed: 2026-04-01, 2025. [Online]. Available: https://www.vinnova.se/en/p/network-enabled-exchange-of-critical-information-in-systems-of-systems-nexsos2/.

[3] A. Sackl, P. Casas, R. Schatz, L. Janowski, and R. Irmer, "Quantifying the impact of network bandwidth fluctuations and outages on web QoE," *2015 7th International Workshop on Quality of Multimedia Experience, QoMEX 2015*, Jul. 2015. DOI: 10.1109/QoMEX.2015.7148078.

[4] M. T. A. Bakar and A. A. Jamal, "Latency issues in internet of things: A review of literature and solution," *International Journal*, vol. 9, no. 1.3, 2020.

[5] O. E. Adebomi and J. S. Mwela, "Impact of packet losses on the quality of video streaming," M.S. thesis, Blekinge Institute of Technology, School of Computing, 2010, p. 40.

[6] T. N. Ananna and M. Saifuzzaman, "Introduction to internet of things," in *IoT and ML for Information Management: A Smart Healthcare Perspective*. Springer Nature Singapore, 2024, pp. 1–49, ISBN: 9789819756247. DOI: 10.1007/978-981-97-5624-7_1. [Online]. Available: http://dx.doi.org/10.1007/978-981-97-5624-7_1.

[7] S. Krishnapriya, B. Manohar, A. Gopi, J. Anna Jacob, R. Mary Puthumana, and A. K. Rizwana, "Implementation of IoT based intelligent and energy efficient cities," in *2022 IEEE 19th India Council International Conference (INDICON)*, 2022, pp. 1–5. DOI: 10.1109/INDICON56171.2022.10039844.

[8] P. Tripathi, A. K. Jha, C. Mahapatra, M. Ansari, and D. K. Mishra, "Smart farming system using IoT, embedded electronics, and wireless sensor networks," in *2025 International Conference on Cognitive Computing in Engineering, Communications, Sciences and Biomedical Health Informatics (IC3ECSBHI)*, 2025, pp. 877–880. DOI: 10.1109/IC3ECSBHI63591.2025.10990435.

[9] D. Bhattacharjee, S. Kumar, Akash, and S. Choudhury, "Design and development of wireless sensor node," *International Journal on Computer Science and Engineering*, vol. 2, Oct. 2010.

[10] H. Kotha and V. Gupta, "IoT application, a survey," *International Journal of Engineering Technology*, vol. 7, p. 891, Mar. 2018. DOI: 10.14419/ijet.v7i2.7.11089.

[11] Z. C. Taysi and A. Yavuz, "Considerations for the design of a general purpose wireless sensor node," *Lecture Notes in Electrical Engineering*, Jan. 2009. DOI: 10.1007/978-0-387-84814-3_63.

[12] F. C. Andriulo, M. Fiore, M. Mongiello, E. Traversa, and V. Zizzo, "Edge computing and cloud computing for internet of things: A review," *Informatics*, vol. 11, no. 4, 2024, ISSN: 2227-9709. DOI: 10.3390/informatics11040071. [Online]. Available: https://www.mdpi.com/2227-9709/11/4/71.

[13] D. Deepak, "Edge computing vs. cloud computing: Complementary or competitive," *International Journal of Applied and Behavioral Sciences*, vol. 02, pp. 65–73, Jan. 2025. DOI: 10.70388/ijabs250109.

[14] National Oceanic and Atmospheric Administration, *What is lidar?* National Ocean Service. Accessed: 2026-06-12, Jun. 2024. [Online]. Available: https://oceanservice.noaa.gov/facts/lidar.html.

[15] VectorNav Technologies, *What is an inertial measurement unit?* Accessed: 2026-06-12. [Online]. Available: https://www.vectornav.com/resources/detail/what-is-an-inertial-measurement-unit-imu.

[16] P. Schulz et al., "Latency critical IoT applications in 5G: Perspective on the design of radio interface and network architecture," *IEEE Communications Magazine*, vol. 55, no. 2, pp. 70–78, 2017. DOI: 10.1109/MCOM.2017.1600435CM.

[17] P. Safir, "Protocols for control and communication in the embedded systems and FPGA," *Universum:Technical sciences*, vol. 114, Sep. 2023. DOI: 10.32743/UniTech.2023.114.9.15958.

[18] J. Chen and S. Huang, "Analysis and comparison of UART, SPI and I2C," in *2023 IEEE 2nd International Conference on Electrical Engineering, Big Data and Algorithms (EEBDA)*, 2023, pp. 272–276. DOI: 10.1109/EEBDA56825.2023.10090677.

[19] Ideas on Board, *Linux UVC driver and tools*, Accessed: 2026-05-20, 2026. [Online]. Available: https://www.ideasonboard.org/uvc/#status.

[20] T. M. Tukade and R. Banakar, "Data transfer protocols in IoT—an overview," *Int. J. Pure Appl. Math*, vol. 118, no. 16, pp. 121–138, 2018.

[21] J. Iyanda, *Connection-oriented TCP vs connectionless UDP: A comparative analysis of network protocols*, Presentation, Accessed: 2026-04-15, May 2020. [Online]. Available: https://www.researchgate.net/publication/370581740_Title_Connection-Oriented_TCP_vs_Connectionless_UDP_A_Comparative_Analysis_of_Network_Protocols.

[22] O. O. Adebola, *Brief insight into transmission control protocol (TCP)*, Research paper, Accessed: 2026-04-20, Jan. 2017. [Online]. Available: https://www.researchgate.net/publication/329337483_Brief_insight_into_transmission_control_protocol_TCP.

[23] J. Postel, "Transmission control protocol," Internet Engineering Task Force, RFC 793, 1981. [Online]. Available: https://www.rfc-editor.org/rfc/rfc793.

[24] S. Zeeshan and S. Parween, "Analysis of TCP issues and their possible solutions in the internet of things," *The International Arab Journal of Information Technology*, vol. 20, Jan. 2023. DOI: 10.34028/iajit/20/2/7.

[25] M. Iglesias-Urkia, A. Orive, and A. Urbieta, "Analysis of CoAP implementations for industrial internet of things: A survey," *Procedia Computer Science*, vol. 109, pp. 188–195, 2017, 8th International Conference on Ambient Systems, Networks and Technologies, ANT-2017 and the 7th International Conference on Sustainable Energy Information Technology, SEIT 2017, 16-19 May 2017, Madeira, Portugal, ISSN: 1877-0509. DOI: 10.1016/j.procs.2017.05.323. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1877050917309870.

[26] S. Hamdani and H. Sbeyti, "A comparative study of COAP and MQTT communication protocols," in *2019 7th International Symposium on Digital Forensics and Security (ISDFS)*, 2019, pp. 1–5. DOI: 10.1109/ISDFS.2019.8757486.

[27] Ericsson, *What is 5G?* Accessed: 2026-05-13, 2026. [Online]. Available: https://www.ericsson.com/en/5g.

[28] M. K. Nishat, O. Gnawali, and A. Abdelhadi, "Adaptive bitrate video streaming for wireless nodes: A survey," *arXiv preprint arXiv:2008.00087*, 2020. DOI: 10.48550/arXiv.2008.00087.

[29] N. Barman and M. G. Martini, "QoE modeling for HTTP adaptive video streaming: A survey and open challenges," *IEEE Access*, vol. 7, pp. 30 831–30 859, 2019. DOI: 10.1109/ACCESS.2019.2901778.

[30] Kubernetes, *Kubernetes components*, Accessed: 2026-06-08. [Online]. Available: https://kubernetes.io/docs/concepts/overview/components.

[31] Kubernetes, *Cluster architecture*, Accessed: 2026-06-08. [Online]. Available: https://kubernetes.io/docs/concepts/architecture.

[32] Kubernetes, *Pods*, Accessed: 2026-06-08. [Online]. Available: https://kubernetes.io/docs/concepts/workloads/pods.

[33] Kubernetes, *Service*, Accessed: 2026-06-08. [Online]. Available: https://kubernetes.io/docs/concepts/services-networking/service.

[34] M. Suznjevic and J. Saldana, "Delay limits for real-time services," *IETF draft*, Jun. 2016.

[35] I. Fathi, Q. Ali, and F. Nazar, "5G latency performance analysis for video streaming applications," *Kufa Journal of Engineering*, vol. 17, pp. 308–329, Feb. 2026. DOI: 10.30572/2018/KJE/170117.

[36] R. Vijay, V. Kumar, R. Pranathi, B. Rajkumar, and B. Amrutur, "Private and public 5g networks for xr use cases: A proof of concept implementations," in *2026 18th International Conference on Communication Systems and Networks (COMSNETS)*, 2026, pp. 865–870. DOI: 10.1109/COMSNETS67989.2026.11418076.

[37] M. Yang, J. Zhang, and L. Yu, "Perceptual tolerance to motion-to-photon latency with head movement in virtual reality," in *2019 Picture Coding Symposium (PCS)*, 2019, pp. 1–5. DOI: 10.1109/PCS48520.2019.8954518.

[38] Arducam, *12MP IMX708 USB3.0 camera datasheet*, Accessed: 2026-05-19, 2023. [Online]. Available: https://blog.arducam.com/downloads/datasheet/B0474_12MP_IMX708_USB3.0_Camera_Datasheet.pdf.

TRITA-EECS-EX-2026:250
Stockholm, Sweden 2026

www.kth.se